



FEDERAL UNIVERSITY OF RIO GRANDE DO SUL
SCHOOL OF ENGINEERING
FINAL PROJECT IN CONTROL AND AUTOMATION
ENGINEERING



Development and Evaluation of a FastSLAM Algorithm in ROS 2: A Case Study for the Capability Enhancement of Robots from Starchê UFRGS Robotics Team.

Author: Gabriel Sadigursky Nunes Cunha

Advisor: Prof. Dr. Rafael Antônio Comparsi Laranja

Co-adviser: Prof. Dr. Renan de Queiroz Maffei

Porto Alegre, June 15, 2026

Contents

Acknowledgments	v
Abstract	vii
List of Figures	ix
List of Abbreviations	xi
List of Symbols	xv
1 Introduction	1
1.1 Motivation	1
1.2 Problem Definition	1
1.3 Objectives	2
1.3.1 General Objective	2
1.3.2 Specific Objectives	2
1.4 Document Structure	3
2 Theoretical Foundation	5
2.1 The Mobile Robot	5
2.1.1 Differential Drive Kinematics	5
2.1.2 Motion Model and Odometry	7
2.2 Perception and Sensors	10
2.2.1 Range Sensors (LiDAR)	10
2.2.2 Measurement Model: Likelihood field model	11
2.3 The Probabilistic Estimation Problem	13
2.3.1 Recursive Bayesian Filtering	14
2.4 Mobile Robot Localization	15
2.4.1 Particle Filters (Monte Carlo Localization - MCL)	16
2.5 Mapping and SLAM	17
2.5.1 Occupancy Grid Maps	17
2.5.2 The SLAM Problem	18
2.5.3 SLAM Approaches	19
3 Tools and Methodology	23
3.1 ROS 2 Ecosystem (Robot Operating System)	23
3.1.1 Architecture: Nodes, Topics, Services	23
3.1.2 Transformation System (TF2)	23
3.2 Simulation Environment	24

3.2.1	Gazebo Simulator	24
3.2.2	RViz2 Data Visualizer	25
3.3	Robot Description (URDF)	26
3.4	Containerization Technology (Docker)	26
3.5	SLAM Repository and Dataset Organization	27
3.6	Evaluation Methodology	27
3.6.1	Map Accuracy	27
3.6.2	Computational Resource Monitoring	27
4	Development and Implementation	29
4.1	Physical Robot Construction (DIY TurtleBot Burger)	29
4.1.1	Component List and Electronic Architecture	29
4.1.2	Mechanical Assembly and Challenges	35
4.2	Hardware-Software Interface	36
4.2.1	Low-Level Hardware Configuration	37
4.2.2	Microcontroller Firmware	37
4.2.3	Operating System and Containerization	37
4.2.4	Hardware Interface ROS2 Nodes	37
4.3	Software Architecture and Implementation	38
4.3.1	The Core Simultaneous Localization and Mapping (SLAM) Node	38
4.3.2	Complementary Nodes	40
4.3.3	Node Structure and Communication Flow	41
4.3.4	Human-Machine Interface (HMI)	42
4.4	Experimental Environment and Navigation Trajectory	48
4.4.1	Environment Design and Structural Features	48
4.4.2	Kinematic Path Planning and Trajectory Execution	50
5	Results and Analysis	53
5.1	Map Generated in the Simulated Environment	53
5.1.1	Simulation Execution and Real-Time Visualization	53
5.1.2	Initial Results with Default Parameters	54
5.1.3	Algorithm Fine-Tuning and Parameter Adjustment	55
5.2	Map Generated with the Physical Robot	56
5.2.1	Physical Robot Execution and Real-Time Visualization	56
5.2.2	Initial Results with Simulated Parameters	57
5.2.3	Sim-to-Real Fine-Tuning and Parameter Adjustment	58
5.3	Quantitative Performance Evaluation	59
5.3.1	Quantitative Map Accuracy Analysis	59
5.3.2	Computational Performance	60
6	Conclusion	63

6.1	Summary of Results	63
6.2	Future Work	64
7	References	65

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my parents, as everything I have achieved is only possible due to the unconditional support I have received throughout my entire life; in particular, I am immensely grateful to my mother for her phenomenal support during the last year marked by the pressures of graduation, the rigorous demands of the university, and my own high expectations.

Furthermore, I extend my thanks to the academic community that supported me throughout this process, especially my advisor and co-advisor, previously acknowledged in this work.

I am also deeply thankful to all the friends I have made along my journey, from the Sinodal robotics team to my university years, as all of you are a part of the passion that has shaped who I am today. Finally, a special thanks to my friend Bruno Richwicki, who not only provided direct assistance in constructing the robot's physical arena but also stood by me as we navigated the challenges of our final academic year.

Abstract

This project presents the development of an open-source FastSLAM algorithm and a mobile robot to provide the Starchê robotics team at UFRGS with a standard framework for autonomous navigation. To address the team's lack of accessible code and hardware, a DIY TurtleBot3 Burger was constructed and upgraded with a Raspberry Pi 5 processing unit. A custom FastSLAM algorithm was then implemented using ROS 2, integrated with a dedicated Human-Machine Interface (HMI) that allows users to easily control either the simulation or the real robot. The entire system was validated first in a Gazebo simulation and subsequently on the physical robot within a custom-built testing arena. The results demonstrated successful mapping in both scenarios. The simulation produced an accurate map with a maximum deviation of 4 cm, while the physical robot generated a functional map with a maximum deviation of 18 cm. Additionally, performance tests confirmed that the chosen hardware efficiently handled the algorithm's computational load without reaching saturation limits. This work successfully delivers a complete, documented baseline for future navigation projects.

List of Figures

1	Global Inertial Frame $\{X_I, Y_I\}$ and Robot Local Reference Frame $\{X_R, Y_R\}$. Adapted from (Siegwart et al., 2011, Fig. 3.1).	5
2	A differential drive robot configuration. Adapted from (Siegwart et al., 2011, Fig. 3.3).	6
3	Detail of a passive caster wheel used for structural stability in differential drive systems. Adapted from (Arduino Forum Contributors, 2021).	7
4	The odometry motion model decomposes relative motion into rotation-translation-rotation. Adapted from (Thrun et al., 2005, Fig. 5.7).	8
5	The odometry motion model distribution $p(x_t u_t, x_{t-1})$ for different noise parameters: (a) Moderate noise, (b) high translational error, (c) high rotational error. Adapted from (Thrun et al., 2005, Fig. 5.8).	9
6	The three components of the likelihood field measurement model. (a) p_{hit} , (b) p_{max} , (c) p_{rand} . Adapted from (Thrun et al., 2005, Fig. 6.2)	12
7	Likelihood field. (a) Example environment with three obstacles (gray) and a sensor measurement indicated by the dashed line. (b) Likelihood field for this obstacle configuration; the darker a location, the less likely it is to perceive an obstacle there. Adapted from (Thrun et al., 2005, Fig. 6.7).	12
8	The composite probability density function for the likelihood field measurement model. It shows the summation of the three components (p_{hit} , p_{max} , p_{rand}) resulting in the final likelihood function. Adapted from (Thrun et al., 2005, Fig. 6.8).	13
9	Gazebo Turtlebot map with Turtlebot3 Burguer Unified Robot Description Format (URDF)	25
10	Gazebo simulation environment displaying the adopted TurtleBot3 Burger URDF.	26
11	Raspberry Pi 5 8GB	30
12	3D printed Waffle Plate used for the DIY TurtleBot3 chassis.	30
13	DIY TurtleBot3 chassis overview highlighting the structural components and the custom Light Detection and Ranging (LiDAR) leveling system.	31
14	Custom 3D-printed wheel hub coupling the Dynamixel motor to the LEGO tire.	32
15	Custom 3D-printed housing and steel bearing ball used as the rear caster wheel.	32
16	DTOF LiDAR LD19 sensor used for environmental scanning.	33
17	Dynamixel XH430-W350-R servomotor used for the robot's actuation system.	33
18	SHIDA 3S 2200mAh LiPo battery used as the centralized power source for the robotic platform.	34
19	Fully assembled DIY TurtleBot3 highlighting the integrated mechanical and electronic components.	36
20	Example of a generated map visualized in RViz2.	40
21	ROS 2 Architecture diagram highlighting nodes, topics, and operational modes.	42
22	HMI initial mode selection window.	43

23	Interface of the Physical Robot control window.	44
24	Interface of the Simulation control window.	45
25	Interface of the Recorded Data control window.	46
26	SLAM Parameters configuration dialog for algorithm fine-tuning.	47
27	Structural layout and dimensional specifications of the experimental navigation arena: (a) 2D top-down ground truth layout with dimensions in millimeters; (b) 3D isometric projection highlighting the partition walls and cylindrical obstacle.	48
28	Visual rendering of the custom experimental arena implemented within the Gazebo simulation environment.	49
29	Computer-Aided Design (CAD) model of the custom 3D-printed support brackets used to secure the internal partition walls.	50
30	The fully assembled physical experimental arena in the Starchê team laboratory.	50
31	Predefined autonomous navigation trajectory designed to ensure complete LiDAR coverage of the experimental arena.	51
32	Real-time execution of the FastSLAM algorithm. Left: RViz2 visualization showing the occupancy grid, LiDAR scans (red dots), odometry (large red arrow), and particles (small RGB axes). Right: Gazebo 3D simulation showing the robot and LiDAR ray-casting (blue beams).	54
33	Initial map generated in RViz2. The structural layout resembles the physical maze, but the map exhibits a noticeable tilt relative to the initial odometry grid.	55
34	Fine-tuned map generated in RViz2. Increasing the rotation noise coefficients and update triggers resolved the tilting discrepancy, aligning the structural walls correctly.	56
35	Real-time execution of the FastSLAM algorithm on the physical robot, visualized in RViz2. The LiDAR scans are represented by multi-colored dots.	57
36	Initial map generated by the physical robot in RViz2 using default parameters. The structural layout partially resembles the physical maze, but the map exhibits a noticeable tilt and distortion relative to the initial odometry grid due to uncompensated real-world drift.	58
37	Final optimized map generated by the physical robot in RViz2.	58
38	Quantitative structural validation via map overlay. The ground truth CAD boundary (red outline) is superimposed onto the generated maps. (a) The simulation map presents a maximum deviation of 4 cm. (b) The physical robot map presents a maximum deviation of 18 cm.	60
39	Hardware resource utilization during the 1000-particle stress test. Both Central Processing Unit (CPU) and Random Access Memory (RAM) consumption remained strictly below theoretical saturation thresholds.	61

List of Abbreviations

AR	Augmented Reality.
CAD	Computer-Aided Design.
CAGR	Compound Annual Growth Rate.
CPU	Central Processing Unit.
DIY	Do It Yourself.
DToF	Direct Time-of-Flight.
EEPROM	Electrically Erasable Programmable Read-Only Memory.
EKF	Extended Kalman Filter.
EM	Expectation-Maximization.
FDM	Fused Deposition Modeling.
HMI	Human-Machine Interface.
LiDAR	Light Detection and Ranging.
LTS	Long-Term Support.
MCL	Monte Carlo Localization.
OS	Operating System.
PaaS	Platform-as-a-Service.
PLA	Polylactic Acid.
RAM	Random Access Memory.
RBPF	Rao-Blackwellized Particle Filter.
REP	ROS Enhancement Proposal.
ROS	Robot Operating System.
SBC	Single Board Computer.
SDF	Simulation Description Format.
SLAM	Simultaneous Localization and Mapping.
SSH	Secure Shell.

TCC	Trabalho de Conclusão de Curso (Final Project).
ToF	Time-of-Flight.
UFRGS	Universidade Federal do Rio Grande do Sul.
URDF	Unified Robot Description Format.
XML	eXtensible Markup Language.
YAML	YAML Ain't Markup Language.

List of Symbols

L	The distance between the wheels [m]. 6
$R(\theta)$	Coordinate transformation matrix. 6
ω	Angular velocity [rad/s]. 7
ξ_I	Robot's pose in the global inertial frame. 5
$bel(x_t)$	Belief distribution of state x_t . 14, 16, 17, 25
l	Half of the distance between the wheels [m]. 6
l_i	Log-odds ratio for cell i . 18
m	Map of the environment. 10, 12, 15, 16, 18, 20, 24, 25
r	Wheel radius [m]. 6
u_t	Control data or odometry at time t . 8, 14, 20, 23, 25
x_t	Robot pose at time t . 8, 10, 12, 14, 15, 18, 25
z_t	Sensor measurement at time t . 10, 11, 14–16, 23, 25
v	Linear velocity [m/s]. 7
M	Total number of particles in the particle filter. 16, 17, 20, 21

1 Introduction

The SLAM paradigm is complex and widely recognized for its implementation challenges (Cadena et al., 2016). It addresses one of the most fundamental problems in robotics, being one of the key enabling technologies of mobile robot navigation. Its functionality arises when a robot is in an unknown place without knowing its own surroundings or location. Just using its odometry measurements, control commands, and surrounding sensing (all of which is noisy data) it is possible to acquire a spatial map while simultaneously approximating its own localization (Thrun et al., 2005; Siciliano and Khatib, 2016).

1.1 Motivation

Although the field has matured significantly in recent decades (Cadena et al., 2016), its importance is still growing at a fast pace and it is already being heavily used in industry. Market research proves that the SLAM market is projected to increase its compound annual growth rate Compound Annual Growth Rate (CAGR) above 28 percent through 2030 (Fortune Business Insights, 2024). This highlights its importance, being a key technology in modern applications like autonomous cars and Augmented Reality (AR). It doesn't come as a surprise that in the academic field, research related to this algorithm has also increased almost exponentially, with thousands of published works annually in the last decade (Gao et al., 2023).

It is undeniable that the explosive growth of the importance of SLAM, both in the market and in the academic field, leads to diverse opportunities for engineering students in the field of robotics. The Mechatronics industry needs integrated skills, which are most effectively taught using Project-Based Learning (Lima et al., 2021). This aligns with the Universidade Federal do Rio Grande do Sul (UFRGS) vision for its university teams, such as the Starchê robotics team (Universidade Federal do Rio Grande do Sul (UFRGS), 2024). Therefore, this Trabalho de Conclusão de Curso (Final Project) (TCC) serves as an ideal opportunity to provide this starting point for the Starchê team, aligning the university's educational goals with this exponentially growing technology.

1.2 Problem Definition

The UFRGS robotics team, Starchê, has a clear gap regarding the use of SLAM technology in their solutions. Although numerous open-source SLAM frameworks exist, the Starchê robotics team lacks a foundational implementation that is fully understood, maintained, and adapted to its own robotic platforms. Developing a SLAM solution from first principles therefore serves not only as a way to obtain a system tailored to the team's needs, but also as an opportunity to build internal expertise and understanding of the underlying algorithms.

Furthermore, while hardware components may be available, the team lacks a standardized, assembled, and documented reference robot platform specifically configured to reliably handle the algorithm. Running SLAM reliably requires hardware capable of providing accurate odometry and

sensor data. The criticality of such data for mobile robot state estimation is highlighted by Lages and Alves (2017) and heavily contextualized in foundational texts (Thrun et al., 2005; Siciliano and Khatib, 2016). Beyond the mechanical aspects, it is also important to mention that SLAM's complexity demands sufficient processing power (Thrun et al., 2005). This creates the need to choose appropriate processing hardware accordingly.

1.3 Objectives

1.3.1 General Objective

To develop a working, open-source SLAM foundational codebase that can be successfully used by the Starchê team. The goal is to validate the implemented SLAM algorithm within a simulated environment and subsequently build a physical robot capable of operating in a planned physical environment, localizing itself, and mapping the space, supported by fully documented code and hardware construction details.

1.3.2 Specific Objectives

The detailed steps for reaching this goal are outlined below:

- Establish the hardware architecture needed to support the chosen SLAM method, taking into account the required processing power and the utilization of currently available components from the Starchê team.
- Create a map design to be utilized in both the simulation and the real-world physical deployment, so it can be used for comparison between the simulation and real-world deployments.
- Implement the SLAM: Implement the core SLAM code, which will be first validated in simulation.
- Test, Validate & Tune SLAM in Simulation: Ensure the code is working correctly in simulation and is ready for use on the real robot.
- Build the Physical Robot: Assemble the real robot based on the selected design and components.
- Prepare Real-World Test Environment: Prepare a physical environment for testing the real robot following the map designed.
- Deploy and Test SLAM on the Real Robot: After validating the code in simulation, run it on the real robot.
- Develop a Human-Machine Interface (HMI): Create a dedicated graphical interface to facilitate the execution of the SLAM code, allowing users to easily switch between simulation, physical and recorded data playback, without needing to manually handle complex node initializations.
- Evaluate Performance & Compare Sim-to-Real: The quantitative evaluation will be performed

by superimposing the generated maps from simulation and Real-world against a ground-truth floor plan to measure structural deviation, alongside an analysis of the computational resource utilization to validate hardware capability.

- Tune the parameters based on real-world testing to improve the robot's performance.
- Structure an open-source codebase repository, allowing comparison across different versions of the SLAM code, and provide a lasting knowledge base for the team.

1.4 Document Structure

This document is organized into six main chapters. Chapter 1 presents the motivation, problem definition, and objectives of the work. Chapter 2 provides the theoretical foundations related to mobile robotics, probabilistic estimation, and SLAM. Chapter 3 describes the tools, simulation environment, and methodology adopted in the project. Chapter 4 details the development and implementation, covering the physical robot construction, the hardware-software interface, and the software architecture. Chapter 5 presents the results and analysis, evaluating the maps generated in both the simulated environment and by the physical robot, along with a quantitative performance evaluation. Finally, Chapter 6 presents the conclusion of this work, summarizing the achievements.

2 Theoretical Foundation

2.1 The Mobile Robot

The following subsections will detail the mobile robot's perception of its environment, focusing on the sensor models used, and then introduce the probabilistic estimation algorithms (specifically Simultaneous Localization and Mapping (SLAM)) that integrate the motion model with the sensor data to achieve accurate navigation.

2.1.1 Differential Drive Kinematics

This section is based on the foundational material on kinematics and differential drive modeling as detailed in: Introduction to Autonomous Mobile Robots (Siegwart et al., 2011, Ch. 3, esp. Sec. 3.2.2), Probabilistic Robotics (Thrun et al., 2005, Ch. 5), and the Springer Handbook of Robotics (Siciliano and Khatib, 2016, Ch.49)

Kinematics is the most basic study of how mechanical systems behave. Understanding the robot's mechanical behavior is necessary for estimating its motion. To model the robot's movement, its position and orientation in the environment are first established. A global Inertial Frame $\{X_I, Y_I\}$ and a Local Reference Frame $\{X_R, Y_R\}$ are defined, attached to a reference point P on the robot chassis, typically the midpoint of the wheel axle (see Figure 1). The robot's pose in the global frame is then given by the vector $\xi_I = [x, y, \theta]^T$, where (x, y) are the coordinates of P and θ is the robot's orientation relative to the X_I axis.

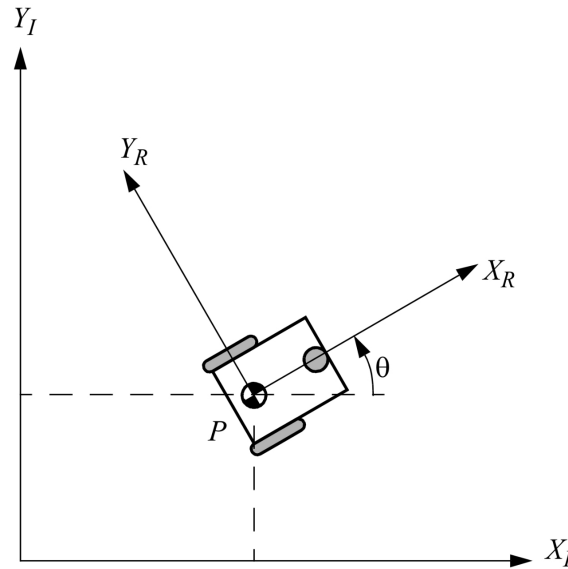


Figure 1: Global Inertial Frame $\{X_I, Y_I\}$ and Robot Local Reference Frame $\{X_R, Y_R\}$. Adapted from (Siegwart et al., 2011, Fig. 3.1).

To transform velocities between the global and local coordinate systems is used the coordinate transformation matrix $R(\theta)$. The velocity vector in the global frame, $\dot{\xi}_I = [\dot{x}, \dot{y}, \dot{\theta}]^T$, can be transformed into the local velocity vector $\dot{\xi}_R = [\dot{x}_R, \dot{y}_R, \dot{\theta}_R]^T$ (which represents the robot's motion relative to the inertial frame, but expressed in the robot's own frame) by multiplying the global vector by the transformation matrix, as shown in Equation 3. To do the opposite, converting local velocity to global, the local vector is multiplied by the inverse of the transformation matrix, $R(\theta)^{-1}$, as seen in Equation 4. The transformation matrix $R(\theta)$ and its inverse $R(\theta)^{-1}$ are defined below:

$$R(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (1)$$

$$R(\theta)^{-1} = R(\theta)^T = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

$$\dot{\xi}_R = R(\theta)\dot{\xi}_I \quad (3)$$

$$\dot{\xi}_I = R(\theta)^{-1}\dot{\xi}_R \quad (4)$$

For a differential drive robot (Figure 2), consisting of two independently driven coaxial wheels and often a passive castor for stability (see example in Figure 3), the forward kinematic model can be derived. This model relates the wheel angular velocities to the robot's linear and angular velocity in its local coordinate frame. The variables used are the wheel radius r , the distance between the wheels $2l$ (also known as wheelbase or track width L), and the angular velocities of the wheels, typically denoted $\dot{\phi}_1$ (right) and $\dot{\phi}_2$ (left). Due to the non-holonomic constraints of this structure, the robot cannot move sideways instantaneously (velocity along the Y_R axis is zero). Forward motion (along the X_R axis) and rotation (around point P) result from the combination of the two wheel velocities.

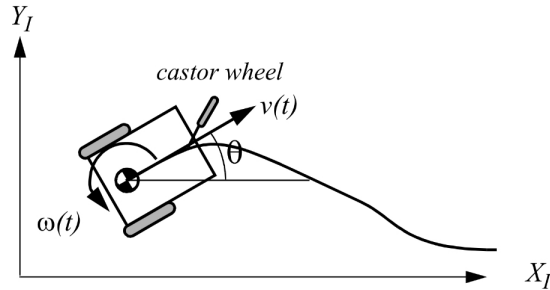


Figure 2: A differential drive robot configuration. Adapted from (Siegwart et al., 2011, Fig. 3.3).



Figure 3: Detail of a passive castor wheel used for structural stability in differential drive systems. Adapted from (Arduino Forum Contributors, 2021).

Assuming no wheel slippage, the robot's linear velocity v along its local X_R axis (forward direction) and its angular velocity ω around the point P are given by the average and difference of the wheel surface speeds:

$$v = \dot{x}_R = \frac{r(\dot{\phi}_1 + \dot{\phi}_2)}{2} \quad (5)$$

$$\omega = \dot{\theta}_R = \frac{r(\dot{\phi}_1 - \dot{\phi}_2)}{2l} \quad (6)$$

The velocity along the local Y_R axis is zero ($\dot{y}_R = 0$). The velocity vector in the local frame is therefore $\dot{\xi}_R = [v, 0, \omega]^T$. Using the inverse rotation matrix (Equation 2), the robot's velocity in the global inertial frame $\dot{\xi}_I = [\dot{x}, \dot{y}, \dot{\theta}]^T$:

$$\dot{\xi}_I = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = R(\theta)^{-1} \dot{\xi}_R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ 0 \\ \omega \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix} \quad (7)$$

Substituting equations (5) and (6) for v and ω , the complete forward kinematic model is obtained

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{r}{2}(\dot{\phi}_1 + \dot{\phi}_2) \cos \theta \\ \frac{r}{2}(\dot{\phi}_1 + \dot{\phi}_2) \sin \theta \\ \frac{r}{2l}(\dot{\phi}_1 - \dot{\phi}_2) \end{bmatrix}. \quad (8)$$

This model relates the wheel velocities directly to the velocities in the global coordinates ideally, without considering slippage. Even without slippage, in practice, robots face uncertainties, which will be discussed in Section 2.1.2 (*Uncertainty and Sensor Noise in Robotic Systems*).

2.1.2 Motion Model and Odometry

This section is based on the principles of Mobile Robot Sensor and Perception Models as detailed in: Probabilistic Robotics (Thrun et al., 2005, Sec. 5.1–5.2, 5.4), the Springer Handbook of

Robotics (Siciliano and Khatib, 2016, Ch.29.1), and Introduction to Autonomous Mobile Robots (Siegwart et al., 2011, Sec. 5.2.4).

The kinematic model derived in Section 2.1.1 describes the ideal motion of a differential drive robot, but when applied in a real robot, the final position after a motion command lies within a distribution of possible poses, not a single, perfect predicted position as calculated before. Real-world robot motion is subject to noise and uncertainty due to factors like wheel slippage, uneven surfaces, and imperfect motor control.

Modeling Motion Uncertainty. The final position of the robot is described by a probabilistic model. The outcome of the control will be described by a posterior probability distribution $p(x_t|u_t, x_{t-1})$, which represents the likelihood of a pose x_t given the previous pose x_{t-1} and the given control u_t . There are two main approaches for this estimation: modeling based on commanded velocities (where the control u_t consists of rotational and translational velocity commands), and modeling based on measured odometry (where the control u_t is derived from the robot's measured odometry poses).

The odometry motion model tends to be more accurate than the velocity motion model, since most commercial robots do not execute velocity commands with the same accuracy as measuring the revolution of the robot's wheels for getting odometry. Although it is possible to fuse both approaches using velocity commands as the motion model and odometry as a sensor this work focuses on the odometry motion model.

The Odometry Motion Model In this model, the motion information u_t is represented by the odometry readings at the start and end of the interval, $\bar{x}_{t-1}$ and $\bar{x}_t$. The relative motion reported by odometry is approximated by decomposing the actual arc trajectory into three steps: an initial rotation (δ_{rot1}), a translation (δ_{trans}), and a final rotation (δ_{rot2}) (Figure 4).

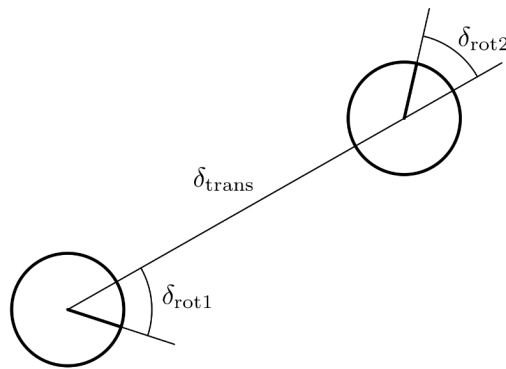


Figure 4: The odometry motion model decomposes relative motion into rotation-translation-rotation. Adapted from (Thrun et al., 2005, Fig. 5.7).

To find the final pose distribution, it is assumed that the true pose is distributed around the odometry-predicted pose with zero-mean noise. The variance of this noise is estimated with robot-specific error parameters. These parameters relate the rotation and translation movements the robot measures to the increase in uncertainty or noise these movements cause. They are (α_1 through α_4) and are tuned by the developer to get the best result for each application of the motion model.

- α_1 : Relates rotational noise to rotational motion.
- α_2 : Relates rotational noise to translational motion.
- α_3 : Relates translational noise to translational motion.
- α_4 : Relates translational noise to rotational motion.

Figure 5 visually demonstrates the effect of the alpha noise parameters on the final pose distribution $p(x_t|u_t, x_{t-1})$. The sub-figures show (a) a typical distribution with moderate noise, (b) high translational error (from α_3, α_4) and (c) high rotational error (from α_1, α_2). This illustrates how the α parameters directly shape the predicted uncertainty cloud.

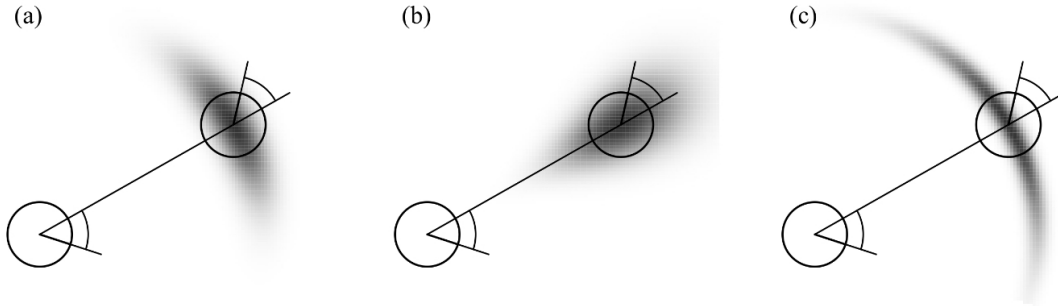


Figure 5: The odometry motion model distribution $p(x_t|u_t, x_{t-1})$ for different noise parameters: (a) Moderate noise, (b) high translational error, (c) high rotational error. Adapted from (Thrun et al., 2005, Fig. 5.8).

These parameters (α_1 through α_4) are used to model the noise-corrupted relative motion represented by $\hat{\delta}_{rot1}$, $\hat{\delta}_{trans}$, and $\hat{\delta}_{rot2}$, which are calculated by sampling from the measured parameters (δ_{rot1} , δ_{trans} , δ_{rot2}) and their associated noise models:

$$\hat{\delta}_{rot1} = \delta_{rot1} - \text{sample}(\alpha_1|\delta_{rot1}| + \alpha_2|\delta_{trans}|) \quad (9)$$

$$\hat{\delta}_{trans} = \delta_{trans} - \text{sample}(\alpha_3|\delta_{trans}| + \alpha_4(|\delta_{rot1}| + |\delta_{rot2}|)) \quad (10)$$

$$\hat{\delta}_{rot2} = \delta_{rot2} - \text{sample}(\alpha_1|\delta_{rot2}| + \alpha_2|\delta_{trans}|) \quad (11)$$

Here, the $\text{sample}(b)$ function generates a random number from a zero-mean distribution (like a Normal or Triangular distribution) with variance b . These equations show how the uncertainty (variance) for each component is a sum of the influences from both rotation and translation, as defined by the α parameters.

The final pose $x_t = [x', y', \theta']^T$ is then calculated by applying the noise-corrupted relative motion $(\hat{\delta}_{rot1}, \hat{\delta}_{trans}, \hat{\delta}_{rot2})$ to the previous pose $x_{t-1} = [x, y, \theta]^T$:

$$\begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} \hat{\delta}_{trans} \cos(\theta + \hat{\delta}_{rot1}) \\ \hat{\delta}_{trans} \sin(\theta + \hat{\delta}_{rot1}) \\ \hat{\delta}_{rot1} + \hat{\delta}_{rot2} \end{bmatrix}. \quad (12)$$

The presented mathematical expressions make it possible both to calculate the probability density $p(x_t|u_t, x_{t-1})$ for a given x_t and to sample a random pose x_t according to this distribution. This odometry motion model forms the basis for the prediction step in many localization and SLAM algorithms, that are going to be more deeply explained in Section 2.5.

2.2 Perception and Sensors

This section uses the theoretical basis of probabilistic robot localization and mapping, notably the framework of the Measurement Model and Beam Models as detailed in: Probabilistic Robotics (Thrun et al., 2005, Ch. 6, Sec. 6.1, 6.3), the Springer Handbook of Robotics (Siciliano and Khatib, 2016, Ch. 5, 31, 46), and Introduction to Autonomous Mobile Robots (Siegwart et al., 2011, Ch. 4, 5).

The subsequent section will synthesize the previously discussed motion model and this measurement model into the overall estimation problem. This will involve introducing the Simultaneous Localization and Mapping (SLAM) framework, detailing how these models are probabilistically integrated to simultaneously track the robot's pose and build a map of its unknown environment.

While the motion model (Section 2.1.2) describes the increase in uncertainty due to the sensors used to measure wheel velocities and the subsequent odometry calculation, this section details a second key component: the Measurement Model. The measurement model, formally $p(z_t|x_t, m)$, describes the probability of receiving a sensor reading z_t given the robot's pose x_t and the map m . As will be further analyzed in Section 2.3, this model decreases the uncertainty, revealing the importance of sensing in a mobile robot.

2.2.1 Range Sensors (LiDAR)

The LiDAR sensor is a common standard for 2D SLAM because it provides fast, dense, and high-resolution range data. Compared to other range sensors, such as sonar, laser range finders are

significantly "more accurate" and suffer from "fewer measurement noise and detection errors". A LiDAR works by emitting a pulse of laser light and measuring the "Time-of-Flight" (Time-of-Flight (ToF)): it calculates the distance to an object by timing how long the light takes to bounce back. By spinning this sensor, it generates a wide (typically 360-degree) "scan" (a point cloud) of the surrounding environment, which is the measurement z_t .

2.2.2 Measurement Model: Likelihood field model

The reading from a LiDAR sensor has several types of errors that can affect the functionality of the pose and mapping approximation, for this reason, just as was done for the motion model, it is necessary to model the measurement data. For that there are more than one approach, the Likelihood field model stand out for offering a Low computational demand and final map smoothness compared to other models (Thrun et al., 2005, Ch. 6, Sec. 6.4), which is a composite model based on three main error sources: measurement noise, failures to detect objects, and random unexplained noise. Each of these components contributes to the final probability distribution and is weighted by a mixing parameter (z_{hit} , z_{max} , and z_{rand} , respectively).

Each of these distributions models a different possible cause for a sensor reading:

- p_{hit} : This is the distribution for a correct measurement. It is modeled as a zero-centered Gaussian (Normal) distribution over the distance between the point measured and the nearest obstacle in the map, accounting for sensor noise (see Figure 6, graph a, where o_1 , o_2 , and o_3 represent physical objects). The probability is evaluated based on the Euclidean distance $dist$, distributed according to $dist \sim \mathcal{N}(0, \sigma_{hit}^2)$. It has one key parameter to be tuned: σ_{hit} (the standard deviation) (Thrun et al., 2005, Ch. 6, Sec. 6.4).
- p_{max} : This models a measurement failure where the sensor reports its maximum possible range, z_{max} (e.g., hitting a black object or an open door). This is modeled as a discrete distribution (see Figure 6, graph b), represented as $I(z_t^k = z_{max})$. It has no tuning parameters, only the sensor's maximum range constant z_{max} .
- p_{rand} : This models a completely random measurement (e.g., crosstalk or a strange reflection). It is modeled as a uniform distribution over the entire sensor range (see Figure 6, graph c). It also has no tuning parameters, as its equation is simply a constant $\frac{1}{z_{max}}$.

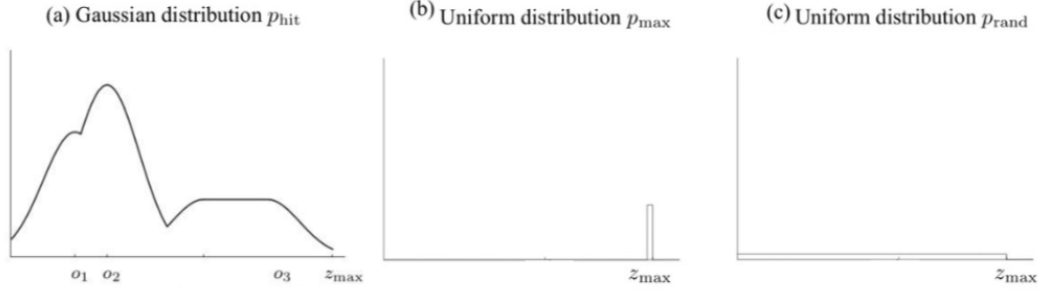


Figure 6: The three components of the likelihood field measurement model. (a) p_{hit} , (b) p_{max} , (c) p_{rand} . Adapted from (Thrun et al., 2005, Fig. 6.2)

The Gaussian distribution of the distance between the measured point and the nearest obstacle used by p_{hit} is calculated with a lookup table generated from the global map m . This table is computed before the evaluation of the measurement model. In practice, this forms a parallel map where each cell stores a probability value based on its distance to the nearest obstacle or wall in the physical space. This computed representation is called the likelihood field, which can be visualized in Figure 7, where brighter areas indicate a higher probability of the sensor detecting an object.

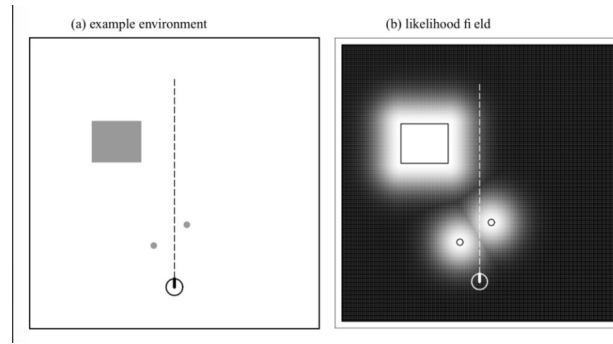


Figure 7: Likelihood field. (a) Example environment with three obstacles (gray) and a sensor measurement indicated by the dashed line. (b) Likelihood field for this obstacle configuration; the darker a location, the less likely it is to perceive an obstacle there. Adapted from (Thrun et al., 2005, Fig. 6.7).

So, for a final estimation of the likelihood of a measurement z_t^k given the state x_t and map m , the measurement model $p(z_t^k|x_t, m)$ is a weighted sum of all the individual component models (p_{hit} , p_{max} , and p_{rand}), each multiplied by its respective mixing parameter (z_{hit} , z_{max} , and z_{rand}).

$$p(z_t^k|x_t, m) = z_{hit} \cdot p_{hit}(z_t^k|x_t, m) + z_{max} \cdot p_{max}(z_t^k|x_t, m) + z_{rand} \cdot p_{rand}(z_t^k|x_t, m) \quad (13)$$

The composite mixture model presented in Equation 13 represents the final probability, or

likelihood, of observing the sensor measurement z_t^k , shown in the Figure 8. This calculated likelihood $p(z_t^k | x_t, m)$ is the necessary component used in the measurement update (correction) step of the Bayes filter, that will be worked in the Section 2.3.1, allowing the robot to accurately weight its pose hypotheses and efficiently reduce uncertainty.

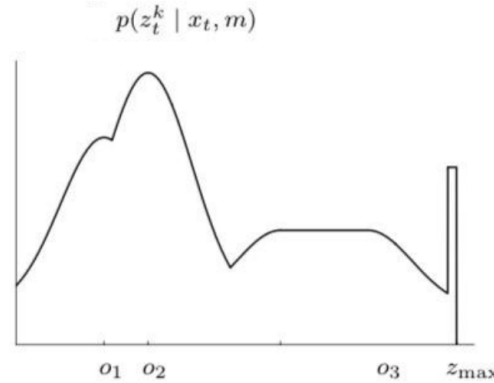


Figure 8: The composite probability density function for the likelihood field measurement model. It shows the summation of the three components (p_{hit} , p_{max} , p_{rand}) resulting in the final likelihood function. Adapted from (Thrun et al., 2005, Fig. 6.8).

2.3 The Probabilistic Estimation Problem

This section uses the foundational concepts of recursive state estimation and the Bayes filter as described in: Probabilistic Robotics (Thrun et al., 2005, p. 9–32), Introduction to Autonomous Mobile Robots (Siegwart et al., 2011, sec. 5.6.7.1), and the Springer Handbook of Robotics (Siciliano and Khatib, 2016, Ch. 5)

The environment of a robot is a dynamic system that possesses an internal state. This internal state has not only map of the environment, but also information about the robot itself. The state is built with all the information that can impact the future states; some important states for this project are cited below:

- The robot pose
- The robot velocity
- The location and features of surrounding objects

A state is called complete when it is the best predictor of the future, and there is no need for knowledge of past states, past control, or measurements, this property is the Markov assumption. These states are modeled depending on past information, so the current state already summarize enough information to make all the past information unnecessary for predicting the next state.

The two fundamental interactions of the robot with its environment are the control and measurement data. Perception is the process to gather information about the state of its environment

resulting in the measurement; the measurements for a specific timestep t will be denoted as z_t . Control data carries all the information about the change of state in the environment. Odometry is treated here as control data as a consequence of the modeling in Section 2.1.2 (although strictly speaking it is sensory data), since when it changes, it represents the change of state of the robot. For a timestep t it will be denoted as u_t . The important relation between the two is that the measurement decreases the uncertainty of the state of the robot. Motion with control signals, on the other hand, tends to increase it.

For getting the best guess of the state in a time step t , it is necessary to have all the information available. In the case of the robot, it would be all the past measurements, control, and states that result in the equation $p(x_t|x_{0:t-1}, z_{1:t-1}, u_{1:t})$, that means the probability of each possible state x_t given all the data available. But following the principles of complete states, the state right before x_t already summarize all the information up to the timestep $t - 1$ making the past information irrelevant. So using the Markov assumption, the final state estimation equation uses only the past state and the new control command given, resulting in a less complex notation $p(x_t|x_{t-1}, u_t)$. Similarly, to get the best assumption of the measurement probability in a certain time t , that also uses all past information with the notation $p(z_t|x_{0:t}, z_{1:t-1}, u_{1:t})$, that can be simplified as $p(z_t|x_t)$.

2.3.1 Recursive Bayesian Filtering

It was explored in the last sections the Motion model and Measurement model, these two concepts are the base for estimating the robot pose in an environment, together they can estimate the state x_t of the robot represented as a belief distribution, or simply belief, with the notation $bel(x_t)$.

The belief is estimated recursively, the first step inside of the loop would be to calculate the prediction using the Motion Model $p(x_t|x_{t-1}, u_t)$ and the belief from the last loop iteration, $bel(x_{t-1})$. This prediction is called intermediate belief and is represented by $\overline{bel}(x_t)$. The second step is the correction step, after moving the robot the uncertainty is increased, so the belief predicted beforehand passes through a correction step using the new measured data.

These two steps known as prediction and correction are from the most general algorithm for calculating beliefs, the Bayes filter, derived in the equations below. It is based on the Bayes rule, which can be used to estimate the distribution of A given a specific measurement distribution B . $p(A|B)$ is known as the Posterior distribution, but it is not possible to get it directly, so it is related to the prior belief $p(A)$ that is the a-priori knowledge of the distribution A , the likelihood $p(B|A)$ that is the probability of observing measurement B given that state A occurred, and finally the probability of the observed data $p(B)$ that usually is used as a normalizing parameter to relate the posterior to the likelihood and prior.

$$p(A|B) = \frac{p(B|A) \cdot p(A)}{p(B)} \quad (14)$$

For the correction step, the equation is very straightforward, the posterior is the corrected belief or the distribution of the state x_t given the measurement z_t , the likelihood is the likelihood of the

measurement z_t given the state x_t , as defined by the already derived measurement model, the Prior belief is just the $\overline{bel}(x_t)$ and the probability of the observed data is used as a normalizer parameter η . The resulting equation is:

$$bel(x_t) = \eta p(z_t|x_t)\overline{bel}(x_t) \quad (15)$$

For deriving the prediction equation, the continuous version of the Law of Total Probability is used, described in the equation below. The integral is a weighted average; this average is the total probability of the data $\mathcal{D}$ being true, considering every possible θ .

$$p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)d\theta \quad (16)$$

For the prediction step, this law is applied by mapping the variables: the desired prediction $\overline{bel}(x_t)$ corresponds to $p(\mathcal{D})$, the motion model $p(x_t|u_t, x_{t-1})$ corresponds to the likelihood $p(\mathcal{D}|\theta)$, and the prior belief $bel(x_{t-1})$ corresponds to the prior $p(\theta)$. The resulting equation is:

$$\overline{bel}(x_t) = \int p(x_t|u_t, x_{t-1})bel(x_{t-1})dx_{t-1} \quad (17)$$

2.4 Mobile Robot Localization

This section is based on the principles of Mobile Robot Localization and Monte Carlo Localization as detailed in: Probabilistic Robotics (Thrun et al., 2005, Sec. 8.3), Introduction to Autonomous Mobile Robots (Siegwart et al., 2011, Chap. 5.3), and the Springer Handbook of Robotics (Siciliano and Khatib, 2016, Ch.13)

In the previous section, the Bayes filter was established as the general method for recursive state estimation. This framework is now applied to one of the most fundamental problems in mobile robotics: Mobile Robot Localization.

Mobile robot localization is the problem of determining the robot's pose, x_t , relative to a given map of the environment, m . As established in Section 2.3, the robot's pose cannot be sensed directly, so it must be inferred by integrating sensor and control data over time.

The straightforward application of the Bayes filter to the localization problem is formally known as Markov Localization. This is not a new algorithm, but rather the same predict-and-correct loop with one important addition: the algorithm is now provided with a map, m . It enters into the correction step in the likelihood $p(z_t|x_t, \mathbf{m})$, and in this way the map is now conditioned on in all calculations since it's a recursive calculation.

There are two main applications of the Markov localization depending on whether the initial position is known.

- Position Tracking, when the initial position is known. It just requires the robot to accommodate

errors from noisy motion, the uncertainty can be represented as a unimodal distribution (a single-peak Gaussian)

- Global Localization, when the initial position is unknown. The robot can be anywhere, so there are different possible distributions for different places in the map the robot could be, for this reason it's a multi-modal distribution.

There are different solutions for solving the two different problems of localization, since this project must handle the global localization problem, a filter is needed that can process complex, multi-modal beliefs. This leads directly to the Particle Filter, a non-parametric implementation of the Bayes filter that is the foundation for Monte Carlo Localization.

2.4.1 Particle Filters (Monte Carlo Localization - MCL)

The previous section concluded that a filter capable of process multi-modal distributions is required to solve the global localization problem. The Particle Filter is a non-parametric implementation of the Bayes filter that perfectly meets this requirement. The specific application of particle filters to the mobile robot localization problem is known as Monte Carlo Localization (MCL).

Instead of representing the belief as a single Gaussian distribution, the particle filter approximates the posterior belief $bel(x_t)$ by a set of M random state samples, called particles.

Each particle $x_t^{[m]}$ is a concrete hypothesis of the robot's pose $[x, y, \theta]^T$. Initially, the particles are randomly spread throughout the known map, and after new control and measurement data come in, the particles are filtered for a better distribution that will consequently guess a better position approximation. The filter is improved through a three-step recursive process implementation of the Bayes filter.

Step 1: Prediction (Sampling) The algorithm first applies the prediction step of the Bayes filter. For each of the M particles $x_{t-1}^{[m]}$ in the previous set, it generates a new hypothetical particle $x_t^{[m]}$ by sampling from the Odometry Motion Model (defined in Section 2.1.2):

$$x_t^{[m]} \sim p(x_t | u_t, x_{t-1}^{[m]}) \quad (18)$$

This step creates a new “cloud” of M predicted particles, $\bar{\mathcal{X}}_t$. The motion model, instead of creating a distribution of all possible positions for each particle, estimates a final position after the control given for a particle based on a random value for noise, so even two particles in the same initial position will have two different final positions, spreading all the particles and incorporating the uncertainty from the robot's movement.

Step 2: Correction (Weighting) Next, the algorithm incorporates the measurement z_t . For each new particle $x_t^{[m]}$, it calculates a numerical importance weight, $w_t^{[m]}$. This weight is the likelihood of the actual sensor measurement z_t given that particle's pose $x_t^{[m]}$ and the map m . If one specific particle is in a location that makes sense given the robot's measurement, its importance weight is high. This value is computed using the LiDAR Measurement Model (defined in Section 2.2.2):

$$w_t^{[m]} = p(z_t | x_t^{[m]}, m) \quad (19)$$

Particles that are in a pose that "agrees" with the sensor data (e.g., matching a hallway in the map) will receive a high weight. Particles in "disagreement" (e.g., in a different room) will receive a low weight.

Step 3: Resampling (Survival of the Fittest) The weighted set of particles $\bar{\mathcal{X}}_t$ now represents the final belief $bel(x_t)$. The last step is to generate a new, unweighted particle set $\mathcal{X}_t$ for the next iteration. This is done by resampling: a new set of M particles is created by drawing M times with replacement from $\bar{\mathcal{X}}_t$. The probability of drawing any specific particle $x_t^{[i]}$ is proportional to its importance weight $w_t^{[i]}$.

This "survival of the fittest" step is what causes the filter to converge. Particles with high weights (good hypotheses) are likely to be duplicated, while particles with low weights (bad hypotheses) are likely to be eliminated. To get a single best-guess pose from the filter at any time, the mean of all M particle positions can be calculated.

2.5 Mapping and SLAM

This section is based on the principles of Simultaneous Localization and Mapping (SLAM), Occupancy Grid Maps, and the FastSLAM algorithm as detailed in: Probabilistic Robotics (Thrun et al., 2005, Chap. 9, 10, 14), Introduction to Autonomous Mobile Robots (Siegwart et al., 2011, Chap. 6, 7), and the Springer Handbook of Robotics (Siciliano and Khatib, 2016, Ch. 13)

This section introduces Simultaneous Localization and Mapping (SLAM), the problem where a robot must build a map of an unknown environment while simultaneously determining its position. It first presents Occupancy Grid Maps as a fundamental method for representing the environment. It then describes the core SLAM problem, explaining the challenge posed by the interdependence between the map and the robot's pose estimate. Finally, it reviews classical SLAM Approaches and introduces FastSLAM, a robust solution that forms the basis of this project.

2.5.1 Occupancy Grid Maps

All the applications discussed for estimating the robot's positions assumed that the map was given in advance, but in cases of SLAM the map is not given so there is the necessity of mapping the environment.

The Occupancy Grid Map is a common and effective non-parametric method for mapping 2D environments. This approach divides the world into a fixed grid of cells, where each cell m_i in the grid can be considered occupied or free. Also simplifying the calculations, occupancy grid mapping makes a strong simplifying assumption: the probability of each cell m_i being occupied is independent of the state of all other cells.

The state of each cell is calculated using the correction equation of the Bayes filter. The map is

fixed; it does not have a cloud of possible different positions like the robot, just a fixed real one, so there is no necessity for a prediction step of the Bayes filter, which predicts the next distribution of possible positions. Basically, $p_t(m_i)$ is the new belief or corrected belief, $p(z_t|m_i, x_t)$ is the measurement model, and $p_{t-1}(m_i)$ is the old belief, or the corrected belief from the last iteration.

$$p_t(m_i) = \eta \cdot p(z_t|m_i, x_t) \cdot p_{t-1}(m_i) \quad (20)$$

However, recursively multiplying thousands of probabilities for each cell could be slow and can lead to numerical errors. To solve this, the algorithm stores the map using the log-odds ratio for each cell m_i . The l_i is just a logarithmic representation of the belief.

$$l_i = \log \frac{p(m_i)}{1 - p(m_i)} \quad (21)$$

A complex Bayesian correction (which uses multiplication) becomes a simple and extremely fast addition. The same correction equation from the Bayes filter changes to this final sum in the logarithmic representation:

$$l_t(m_i) = l_{t-1}(m_i) + \log \frac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} \quad (22)$$

This correction, however, requires an inverse measurement model $p(m_i|z_t, x_t)$. That, just like in the correction equation for the Bayes filter derived in Section 2.3, is derived using the measurement model $p(z_t|x_t)$, more specifically the forward model $p(z_t|m_i, x_t)$. Since this equation, when made for each cell in a grid, can be computationally expensive and also slow for when the robot is running, a common technique to avoid this is to calculate the inverse measurement model using the forward model for every possible robot state and measurement before the robot starts running, and put this information in a lookup table. When the robot is running and finds itself in one specific state and has one specific measurement, it can gather the probability of the sensed cell being occupied from this table without needing to calculate it from zero, enabling a fast iteration of the correction.

The final map is built by analyzing each cell's probability of being occupied, separating each cell into one of three states: occupied (high probability), free (low probability), or unknown (typically 0.5 probability).

2.5.2 The SLAM Problem

The previous sections have treated localization and mapping as separate problems:

- In Mobile Robot Localization (Section 2.4), the map m was assumed to be given and correct, and the robot's pose x_t was estimated.
- In Occupancy Grid Maps (Section 2.5.1), the robot's pose x_t was assumed to be given and correct, and the map m was estimated.

The SLAM problem arises when a robot must build a map of an unknown environment while simultaneously tracking its own position within that map, using only its control inputs $u_{1:t}$ and sensor measurements $z_{1:t}$. To build an accurate map, the robot must know its pose and to get an accurate pose, the robot must have an accurate map to localize against, this codependency is what makes SLAM a difficult algorithm to be implemented. A small error in the pose estimate will cause the map to be built incorrectly. This incorrect map will then be used in the next correction step, which in turn will cause an even larger pose error. The errors in pose and map compound each other.

Another challenge is the high-dimensional state space necessary for SLAM, as it includes the robot's pose and the coordinates of every feature in the map, which can be tens of thousands of variables. To retroactively correct the map during the run, it is necessary to keep track of the all past information, this way every new information can help correct not only the newest state but also the past states, and it is also required to solve the correspondence problem. This correspondence problem is complex because, given the robot's uncertainty, a measurement can appear equivalent to more than one location on the map. Without keeping the history of all past information, a wrong deduction about the robot's location (a correspondence error) leads to an uncorrectable error.

2.5.3 SLAM Approaches

Solving the SLAM problem requires algorithms that can manage the high-dimensional state space and the data correspondence problem. The probabilistic approaches to solving this are based on the Bayes filter, leading to two main families of algorithms.

The first and most historically influential solution is the Extended Kalman Filter (EKF). This approach solves SLAM by creating one single, massive state vector. This vector includes both the robot's pose $[x, y, \theta]^T$ and the estimated $[x, y]$ coordinates of every landmark found in the map. The filter also maintains one giant covariance matrix that stores the uncertainty of every variable and the interaction between the variable uncertainty. This covariance is what makes EKF SLAM work: when the robot re-observes a landmark (closes a loop), the filter updates the robot's pose, and this update then propagates through the covariance matrix since the covariance contains the relation between each variable, correcting the poses of all other landmarks in the map.

While foundational, EKF SLAM has critical limitations. Its computational cost is $O(N^2)$, quadratic in the number of landmarks N , due to the storage of each relation between the variables, which makes it computationally impossible to use for large maps. Also, because it is unimodal, this solution cannot solve the global localization problem.

Another approach for solving SLAM is the Expectation-Maximization (EM) algorithm. This is an offline method that takes the data from a complete run and recursively polishes it to get the best approximation of a map. It works by iterating two steps: an Expectation step, where it calculates the most likely path for the robot given its current noisy map gotten from the run, and a Maximization step, where it uses that path created from the last step to create a new, most likely map. It repeats these two steps recursively until the map and path converge.

The main disadvantage of EM SLAM is that, as an offline method, it must have all the sensor

and control data from the entire run, making it unusable for a robot that needs to build a map in real-time.

These limitations, the $O(N^2)$ cost and fragility of EKF SLAM, and the offline nature of EM SLAM, led to the development of a hybrid algorithm that is both robust and fast enough for online, real-time mapping. This algorithm, which forms the basis of this project, is known as FastSLAM.

FastSLAM is based on the Rao-Blackwellized Particle Filter (RBPF). Its core principle is to solve the SLAM problem by decomposing the high-dimensional joint posterior into two simpler, manageable components. The goal of the full SLAM problem is to estimate the joint posterior distribution of the robot's trajectory $x_{1:t}$ and the map m , given all observations $z_{1:t}$ and controls $u_{1:t}$:

$$p(x_{1:t}, m | z_{1:t}, u_{1:t}) \quad (23)$$

Using the chain rule of probability, this distribution can be factored into the product of the robot's path posterior and the map posterior conditioned on that path:

$$p(x_{1:t}, m | z_{1:t}, u_{1:t}) = \underbrace{p(m | x_{1:t}, z_{1:t})}_{\text{Mapping}} \cdot \underbrace{p(x_{1:t} | z_{1:t}, u_{1:t})}_{\text{Localization}} \quad (24)$$

This factorization allows FastSLAM to solve the two parts using different algorithms tailored to their specific characteristics. The localization component $p(x_{1:t} | z_{1:t}, u_{1:t})$ is estimated using a particle filter, where each particle represents a hypothesis of the robot's entire trajectory. The mapping component $p(m | x_{1:t}, z_{1:t})$ is then solved analytically using the Occupancy Grid method, conditioned on the path provided by each particle.

For the next derivations, it is important to know that each particle builds its own map and moves by its own path hypothesis. It is a possible solution for both mapping and localization. Just like the original particle filter, it is a complete guess of the real robot's state, but instead of just being a good guess for localization, it is also a guess for the map.

For the localization problem, the first step is just like the Prediction for the standard particle filter. For each of the M particles, a new pose $x_t^{[k]}$ is sampled from the Odometry Motion Model (Section 2.1.2) based on the previous pose $x_{t-1}^{[k]}$ and the control u_t . This new pose is then added to that particle's path history $x_{1:t}^{[k]}$.

$$x_t^{[k]} \sim p(x_t | x_{t-1}^{[k]}, u_t) \quad (25)$$

The second step is Correction and Weighting. At first glance, it is very similar to the correction step of the particle filter. It uses the measurement model and the map that each particle is currently building to find if the measurement made by the robot corresponds to that particle's map. If a particle's movement is deviating too much from the robot's actual movement, the measurement received by the real robot is not going to make sense in this specific particle's map, causing it to

have a low likelihood. This way, only the particles that are moving in accordance with the robot will have a good likelihood.

At the same time as the weighting, the mapping step occurs: each particle runs the log-odds technique for its own occupancy grid, correcting its map based on the new measurement.

The third step is Resampling. This step is identical to MCL. A new set of M particles is created by sampling with replacement from the old set, where the probability of any particle k being “cloned” is proportional to its weight $w_t^{[k]}$. When a particle is cloned, its full path history and its entire occupancy grid map are duplicated.

Since particles that are moving more according to the robot’s movement have a better likelihood and will more likely be resampled, a pretty good approximation of the robot’s real localization is obtained. And since these good particles have their own occupancy grid mapping happening, and their localization is good, the map built is also good.

This hybrid structure directly solves the problems of EKF SLAM. It is robust to data association errors because an incorrect correspondence will only corrupt a few particles, which will then receive low weights and be eliminated during resampling. It can solve the global localization problem because it is a particle filter. Finally, it scales much better. The $O(N^2)$ cost of EKF SLAM is replaced by an algorithm whose cost scales with the number of particles M , making it far more efficient for large maps.

3 Tools and Methodology

To implement SLAM in a scalable and accessible project, it was chosen to first simulate a robot using the Robot Operating System (ROS) 2, a software framework specifically designed to meet the demands of modern robotics (Macenski et al., 2020). With this software as the core of the project, it was chosen to build a robot similar to the TurtleBot3 Burger, a platform specifically designed for use with ROS (ROBOTIS, 2024d).

3.1 ROS 2 Ecosystem (Robot Operating System)

The Robot Operating System (ROS 2) provides a standardized framework for robotic software development (Macenski et al., 2020). This project utilizes the ROS 2 Humble Hawksbill distribution, running natively on Ubuntu 22.04 (Canonical Ltd., 2024), to ensure long-term stability. It supplies the communication infrastructure to integrate physical sensors and actuators with SLAM algorithms. To clarify this integration, the following subsections outline the core ROS 2 communication architecture and the TF2 transformation system responsible for managing the robot's coordinate frames.

3.1.1 Architecture: Nodes, Topics, Services

The ROS 2 architecture is based on a distributed graph of processes, called Nodes, which are independent executables. These nodes communicate with each other using three primary mechanisms: Topics, Services, and Actions (Alonso, 2025, Sec. 2.2.1).

Topics: Topics are the primary communication method for the continuous data streams. They use an asynchronous, many-to-many "publisher/subscriber" model (Robotics, 2022b). One node (a publisher) sends a message to a topic (e.g., `/scan`), and any node that subscribes to that topic receives the message. This mechanism is how the sensor data (z_t) and odometry data (u_t) are transmitted to the main SLAM node.

Services: Services are used for synchronous, request/response communication. They follow a one-to-one "client/server" model, where a client node sends a request and waits for a response from a server node (Robotics, 2022a). Unlike topics, services are for quick, atomic transactions.

Actions: Actions are designed for long-running, asynchronous tasks that require feedback. Like services, they use a client/server model, but the client does not have to wait for the task to finish. An action client sends a "goal" (e.g., `NavigateToPose`) to an action server, which then executes the task and provides a continuous stream of feedback (e.g., current distance to the goal) and a final result. This allows a task to be monitored and, if necessary, canceled (Alonso, 2025, Sec. 2.2.1).

3.1.2 Transformation System (TF2)

Robotic systems must frequently track complex spatial relationships between many coordinate frames, such as the location of sensors relative to the robot's base, or the robot's pose relative to a

fixed world reference (Alonso, 2025, Sec. 2.4). To simplify and unify the handling of these frames, ROS 2 uses the TF2 transformation system (Foote, 2013). This system constructs a dynamic tree of transformations that relates all coordinate frames present in the system (Alonso, 2025, Sec. 2.4).

In the context of mobile navigation, TF2 is essential for managing pose estimation and odometry data fusion. The standard transformation architecture, defined by ROS Enhancement Proposal (REP) 105 (Community, 2010), establishes three primary coordinate frames to manage the difference between the ideal pose and real-world drift, which are then supplemented by specific sensor frames for hardware integration (Alonso, 2025, Sec. 2.9):

- **map:** A fixed, global coordinate frame. This frame is the origin of the map m and is considered to be the "true" world. The pose of the robot in this frame should not drift.
- **odom:** A fixed frame that represents the robot's starting position. The pose of the robot relative to the odom frame is published by the odometry system (e.g., from encoders or visual odometry). This transform, from the odom frame to the base_link frame, is continuous, but it accumulates error and will "drift" over time (Alonso, 2025, Sec. 2.9).
- **base_link:** The main coordinate frame attached to the robot's mobile base.
- **Sensor Frames (e.g., laser_frame):** Coordinate frames attached to specific sensors mounted on the robot. This ensures that sensor measurements (like LiDAR scans) are accurately projected into the robot's core coordinate system before being evaluated by the SLAM algorithm.

The fundamental task of the SLAM algorithm (described in Section 2.5.3) is to calculate and publish the correction that compensates for the odometry drift. It does this by continuously broadcasting the transformation between the map frame and the odom frame.

3.2 Simulation Environment

Simulation plays a fundamental role in the development and validation of robotic systems, allowing for controlled, safe, and reproducible tests to be performed before real-world application (Alonso, 2025, Sec. 2.1). For a probabilistic algorithm like SLAM, this approach provides a critical advantage not available in the physical world: access to absolute "ground truth" data. This project adopts two Simulation central ROS 2 tools: the Gazebo simulator for physics and sensor generation, and RViz2 for data visualization.

3.2.1 Gazebo Simulator

Gazebo is a high-fidelity, open-source 3D physics simulator. Unlike a simple visualizer, Gazebo simulates a complete physical world, including gravity, friction, and sensor physics (Koenig and Howard, 2004). This allows for the development and testing of the entire SLAM algorithm before the physical Do It Yourself (DIY) robot hardware is assembled, decoupling software development from hardware integration. The simulation that is gonna be used is custom build map, with the Turtlebot3 Burger robot model, shown in the Figure 9.

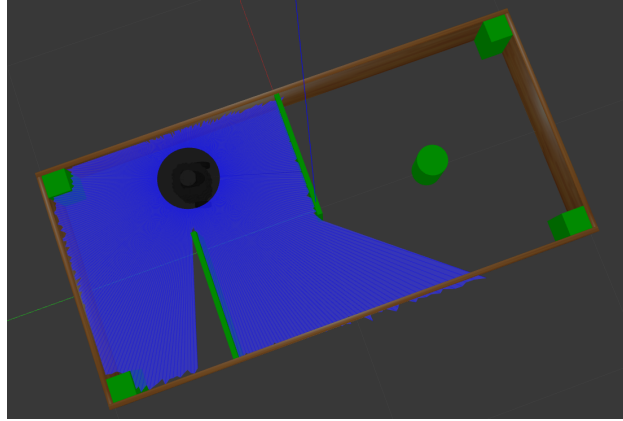


Figure 9: Gazebo Turtlebot map with Turtlebot3 Burguer URDF

Source: The Autor

In the context of this project, Gazebo’s primary role is to generate the realistic sensor and control data streams that the SLAM algorithm will consume.

- **Physics Simulation:** Gazebo simulates the robot’s differential drive, including joint forces, friction, and inertia. When a control command is sent, the simulator’s physics engine generates a realistic odometry message (u_t), which inherently includes the types of noise and drift (like wheel slippage) described in the Motion Model (Section 2.1.2).
- **Sensor Simulation:** A Gazebo plugin is used to simulate the LiDAR sensor. This plugin performs virtual ray-casting against the 3D world model and generates a sensor message (z_t) that mimics the real sensor. This simulated data includes noise, mimicking the p_{hit} , p_{max} , and p_{rand} components of the Measurement Model (Section 2.2.2).

This allows the SLAM algorithm to be developed in an environment that is a close approximation of the real world (Alonso, 2025, Sec. 2.1).

3.2.2 RViz2 Data Visualizer

RViz2 is the primary 3D visualization tool for ROS 2. It subscribes to any ROS topic and displays its contents, making it the main debugging interface for the SLAM algorithm.

For this project, RViz2 is used to visualize:

- The Occupancy Grid Map (m) being published by the SLAM node.
- The particle cloud ($bel(x_t)$) from the FastSLAM algorithm.
- The raw sensor data (z_t) from the `/scan` topic.
- The TF2 transformation tree (Section 3.1.2), which shows the final, corrected pose x_t of the robot on the map.

3.3 Robot Description (URDF)

Since this project involves a construction of an existing robot, a standard robot description file is already available. The URDF of the existing robot can be seen in the 10. It defines:

- **Links and Joints:** The physical components of the robot (e.g., base_link, left_wheel, laser_frame) and the joints that connect them (e.g., left_wheel_joint).
- **Physics and Collisions:** The mass, inertia, and collision geometry of each link, which Gazebo uses for realistic simulation (Alonso, 2025, Sec. 2.5).
- **Static Transforms:** The URDF is used to automatically generate the static TF2 transforms as described in the section 3.1.2

Although this project involves a DIY version of the TurtleBot3 Burger platform, a complete robot description file does not need to be written from scratch. Instead, the official TurtleBot3 URDF, publicly provided by ROBOTIS, is adopted as a baseline model, as shown in Figure 10 (ROBOTIS, 2024d).

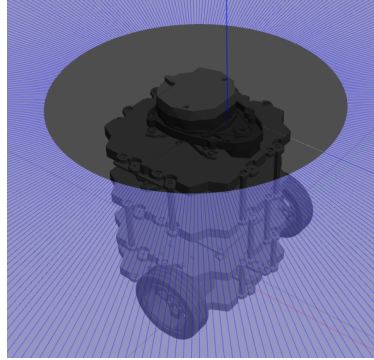


Figure 10: Gazebo simulation environment displaying the adopted TurtleBot3 Burger URDF.

Source: The Author

3.4 Containerization Technology (Docker)

To ensure software consistency across different hardware platforms that will be used in this project, Docker was utilized as a core deployment tool. Docker is a Platform-as-a-Service (PaaS) product that uses Operating System (OS)-level virtualization to deliver software in packages called containers. These containers isolate the application's environment, ensuring that the ROS 2 Humble ecosystem runs with its exact dependencies (such as the required Ubuntu 22.04 base image) regardless of the host operating system (Matthias and Kane, 2015). This tool is fundamental for the methodology of this project, as it allows the SLAM algorithm to be seamlessly transitioned from the local development computer to the physical embedded hardware without version conflicts or dependency degradation.

3.5 SLAM Repository and Dataset Organization

A key contribution of this work is a structured, open-source repository hosted on GitHub (Cunha, 2026b). It provides a centralized base for the developed software, tests, and results.

The repository is divided into three main categories:

- **Source Code and Configuration:** The C++ FastSLAM node, the Python Control Station (HMI), Docker deployment files, ROS 2 parameters (YAML Ain't Markup Language (YAML)), URDF models, and launch scripts. The source code features inline comments cross-referencing specific sections of this document, enabling students to directly connect the practical implementation with the theoretical foundation.
- **Datasets:** ROS 2 bag files (.db3) storing sensor and odometry data from both the Gazebo simulation and the physical robot experiments.
- **Generated Maps:** Final occupancy grids (.pgm and .yaml) alongside metadata such as map resolution.

Being fully open-source (Cunha, 2026b), this repository not only serves the Starchê UFRGS Robotics Team but also enables other universities and researchers to build the physical platform, utilize the simulation environments, and contribute system improvements.

3.6 Evaluation Methodology

To validate the performance of the FastSLAM algorithm, two evaluation metrics were established. This methodology ensures an objective measurement of the transition from the simulated environment to the physical robot.

3.6.1 Map Accuracy

To evaluate the accuracy of the generated occupancy grids, the theoretical ground truth (the CAD floor plan detailed in Section 4.4) is superimposed onto the generated maps. The structural deviation is calculated by counting the divergent pixels between the generated walls and the ground-truth outline. By multiplying this pixel count by the map resolution the absolute metric error of the map is obtained.

3.6.2 Computational Resource Monitoring

To verify that the selected embedded hardware can handle the algorithm, CPU and RAM consumption inside the Docker container are recorded at 1 Hz during execution. The hardware is considered sufficient if peak utilization remains safely below saturation thresholds, preventing data throttling and ensuring real-time performance.

4 Development and Implementation

This chapter details the practical development and experimental deployment of the robotic platform. The implementation is organized into three main pillars: the physical construction of the custom DIY TurtleBot3, the integration of the hardware with the ROS 2 ecosystem, and the orchestration of the FastSLAM algorithm through a custom graphical interface. Each section describes the specific hardware choices, the software architecture used to bridge low-level control with high-level mapping, and the design of the experimental environment used for final validation.

4.1 Physical Robot Construction (DIY TurtleBot Burger)

The physical robot was built to be as close as possible to the original TurtleBot3 Burger. To achieve this, the official hardware list found in (ROBOTIS, 2026) was used as a baseline. The main constraint was the budget available for the build. Consequently, for each key part, the closest functional alternative within the budget was chosen. However, the original main processing unit lacks the computational capacity required to run the FastSLAM algorithm effectively. Because of this limitation, an enhanced processing unit was selected. The specific technical reasons for this upgrade, along with the justification for all other selected components, are detailed in the following subsections

4.1.1 Component List and Electronic Architecture

Deploying the simulated system into the real world requires a robust hardware architecture. This section outlines the component list and electronic integration of the custom DIY TurtleBot3. It details the processing unit, mechanical chassis, sensors, actuation, communication, and power distribution systems required to support the FastSLAM algorithm.

Main Processing Unit (Single Board Computer (SBC)): The original hardware architecture of the TurtleBot3 Burger utilizes a Raspberry Pi as its Single Board Computer (SBC). Depending on the manufacturing generation, it is equipped with either a Raspberry Pi 3 (featuring a 1.2GHz processor and 1GB of RAM) or a standard Raspberry Pi 4 (featuring a 1.5GHz processor and options ranging from 2GB to 4GB of RAM) (ROBOTIS, 2026). However, Simultaneous Localization and Mapping (SLAM) algorithms are highly resource-intensive, as previously established in Section 2.5.3.

Current deployment guidelines for TurtleBot3 SLAM applications explicitly recommend an SBC with at least 4GB of RAM (Zbotic, 2024). Furthermore, experimental reports from the robotics community indicate that even standard Raspberry Pi 4 boards can experience processing bottlenecks and memory constraints, struggling to keep up with the data rates of standard LiDAR sensors during SLAM execution (Exchange, 2025). To address these challenges and ensure sufficient computational headroom for the FastSLAM implementation, the Raspberry Pi 5 with 8GB of RAM (Figure 11) was selected for this project (Raspberry Pi Ltd, 2023). This hardware upgrade provides the necessary processing speed and memory capacity to handle multiple particles and high-frequency sensor data

without performance degradation.

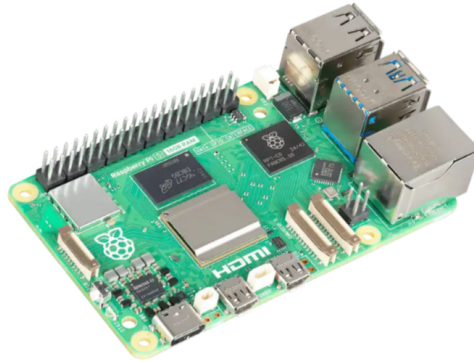


Figure 11: Raspberry Pi 5 8GB

Source: <https://www.raspberrypi.com>

Mechanical Chassis: The structural foundation of the original TurtleBot3 Burger relies on injection-molded plastic components known as Waffle Plates. The DIY physical chassis was entirely manufactured using 3D printing. An open-source, dimensionally accurate 3D model of the Burger chassis, officially provided by (ROBOTIS, 2018), was utilized. The parts were fabricated using Fused Deposition Modeling (FDM) technology on an Anycubic i3 Mega 3D printer. To ensure the high mechanical strength to support the processing unit and to minimize sensor vibrations, the chassis was printed using Polylactic Acid (PLA) filament with a 100% solid infill. The manufacturing parameters were configured with a 0.4 mm nozzle diameter and a 0.3 mm layer height.



Figure 12: 3D printed Waffle Plate used for the DIY TurtleBot3 chassis.

Source: The Author

A unique modification to the original design was the inclusion of an additional custom-designed top plate to house the LiDAR sensor (Cunha, 2026a). This layer features a four-point kinematic leveling system, which utilizes spring-loaded fasteners to provide adjustable planar alignment. In

2D SLAM algorithms, the environment is mapped based on a strict planar assumption, requiring the sensor's measurement plane to be strictly parallel to the ground (Thrun et al., 2005). Any uncompensated pitch or roll misalignment causes the laser beams to either intersect the floor or scan above low-profile objects. Therefore, this leveling implementation allows for the fine-tuning of the sensor's orientation, mitigating severe mapping errors and ensuring the high fidelity of the spatial data fed to the FastSLAM algorithm.

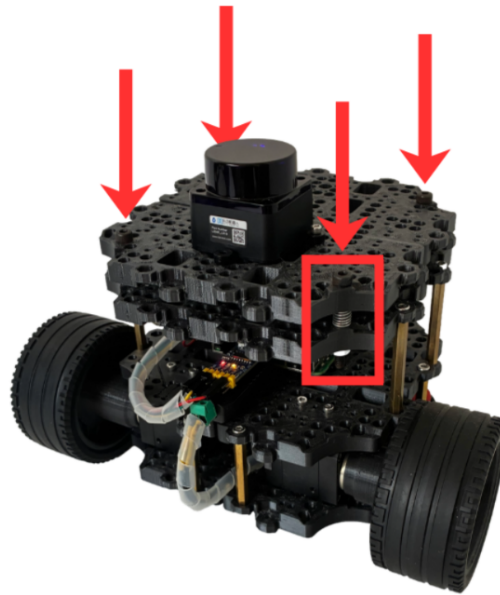


Figure 13: DIY TurtleBot3 chassis overview highlighting the structural components and the custom LiDAR leveling system.

Source: The Author

Wheels and Actuation: The standard TurtleBot3 Burger utilizes Dynamixel XL430-W250-T servomotors (ROBOTIS, 2024c). For this project, the actuation system was upgraded to the high-performance Dynamixel XH430-W350-R servomotors (ROBOTIS, 2024b). This upgrade ensures zero performance loss compared to the standard TurtleBot3 simulation, as both models share the same contactless magnetic encoder with a resolution of 4096 steps per revolution, maintaining identical odometry transform (tf) parameters. Furthermore, the XH430-W350-R is technically superior due to its full-metal internal gears, a significantly higher starting torque of 3.4 N.m (compared to the original 1.5 N.m), and the addition of current-based control for real-time torque monitoring.

To translate this power into movement, LEGO 44771 tires were coupled to the motors via a custom 3D-printed wheel hub (BrickLink, 2024). While the original wheels have a 66 mm diameter

and a 19 mm width, the LEGO tires feature a larger diameter of 68.8 mm and a significantly wider 36 mm flat profile. These dimensions directly affect the robot's kinematics, as the 4.2% increase in diameter means the robot travels more distance per motor revolution than the standard model. To prevent cumulative positioning errors in the FastSLAM algorithm, this discrepancy is corrected via software by updating the wheel radius parameters, ensuring that the physical odometry remains synchronized with the simulation environment.



Figure 14: Custom 3D-printed wheel hub coupling the Dynamixel motor to the LEGO tire.

Source: The Author

Passive Caster Wheel: To ensure the robot's stability through a three-point contact system following the original model, a passive caster was implemented at the rear of the chassis. A repurposed automotive steel bearing ball with a diameter of 11.5 mm was utilized. A custom 3D-printed housing was specifically modeled to act as a socket, allowing the ball to rotate freely in any direction with minimal friction while maintaining constant contact with the ground. This ball caster mechanism provides the necessary structural support for the rear section of the robot, ensuring smooth omnidirectional maneuverability and maintaining the horizontal alignment required for consistent LiDAR data acquisition.



Figure 15: Custom 3D-printed housing and steel bearing ball used as the rear caster wheel.

Source: The Author

Sensors (LiDAR and Odometry) :Due to component availability, the DTOF LiDAR LD19 was selected as the primary range sensor. This model utilizes Direct Time-of-Flight (Direct Time-of-Flight (DToF)) technology to provide continuous 360-degree two-dimensional environmental scans.

It generates the dense point cloud data, representing the measurement z_t , which is crucial for the correction step of the probabilistic estimation (Thrun et al., 2005, Ch. 6).



Figure 16: DToF LiDAR LD19 sensor used for environmental scanning.

Source: LDROBOT (2022)

For odometry, the system relies on the built-in contactless magnetic encoders of the upgraded Dynamixel XH430-W350-R servomotors (ROBOTIS, 2024b). These encoders provide a high resolution of 4096 steps per revolution, enabling precise tracking of individual wheel rotations (ROBOTIS, 2024b). This low-level data is subsequently processed through the differential drive kinematic model to calculate the robot's relative motion u_t over time, feeding the prediction step of the localization filter (Thrun et al., 2005, Sec. 5.1).



Figure 17: Dynamixel XH430-W350-R servomotor used for the robot's actuation system.

Source: ROBOTIS (ROBOTIS, 2024b)

Communication Interfaces (RS485 and UART to USB) : To establish a reliable data link between the high-level processing unit (Raspberry Pi 5) and the low-level hardware components, specific serial communication bridges were implemented. The Dynamixel XH430-W350-R actuators operate on a half-duplex RS-485 serial network (ROBOTIS, 2024b). Because the Single Board

Computer lacks native RS-485 ports, dedicated RS485-to-USB converters, featuring FT232 logic chips, were employed. For this hardware configuration, an independent converter was assigned to each motor, connecting them directly to separate USB ports on the processing unit. Similarly, the LD19 LiDAR utilizes a dedicated UART-to-USB serial module to transmit its dense point cloud data (ShenZhen LDROBOT Co., LTD, 2022a). This interface setup ensures that both the proprioceptive and exteroceptive sensor pipelines are physically integrated and seamlessly exposed to the ROS 2 environment for the FastSLAM node.

Power Distribution System :The power distribution system was designed to ensure electrical stability while utilizing a single, centralized power source. The system is powered by a SHIDA 3-cell (3S) LiPo battery with a nominal voltage of 11.1V and a capacity of 2200mAh. From this main power source, the circuit splits into two branches. The first branch provides a direct connection to the Dynamixel XH430-W350-R servomotors, leveraging the battery's voltage to supply their actuation requirements (ROBOTIS, 2024b). The second branch provides power to the Raspberry Pi 5 through an XL4015-based step-down (buck) converter, specifically the HW-064 module, which also helps mitigate voltage sags reaching the processing unit during motor load spikes (XLSEMI, 2012).



Figure 18: SHIDA 3S 2200mAh LiPo battery used as the centralized power source for the robotic platform.

Source: The Author

To mathematically validate this architecture, the combined maximum power draw of the processing unit and the actuation system was calculated. The Raspberry Pi 5 requires a 5.1V supply and can draw up to 5A under maximum processing load, resulting in a peak power requirement of approximately 25.5 Watts ($P = V \times I$) (Raspberry Pi Ltd, 2023). Similarly, each Dynamixel XH430-W350-R servomotor has a maximum stall current of 1.4A (ROBOTIS, 2024b). For the two-wheel differential drive configuration, the total peak current draw from the motors reaches 2.8A, corresponding to a maximum power demand of roughly 33.6 Watts (at an assumed 12V). Therefore, the entire robotic platform presents a combined peak power demand of 59.1 Watts (25.5W + 33.6W).

The dedicated SHIDA LiPo pack has a nominal voltage of 11.1V (reaching up to 12.6V when

fully charged) and a capacity of 2200mAh (2.2Ah). To supply the 59.1W peak demand at the nominal 11.1V, the battery must provide a total continuous current of approximately 5.32 Amps ($I = P/V$). For a 2.2Ah battery, this current draw represents a required discharge rate of merely 2.4C (5.32A/2.2Ah). Since standard commercial robotics and UAV LiPo batteries safely support continuous discharge rates upwards of 15C to 20C (Boukoberine et al., 2019), the combined peak demand of 5.32A is well within the safe operational limits of the component, ensuring an extremely stable energy source for the system.

4.1.2 Mechanical Assembly and Challenges

The main challenges during the mechanical assembly involved circuit soldering and power routing. For the battery and main power connections, XT60 connectors were utilized.

Powering the Raspberry Pi 5 presented a specific challenge. After evaluating various methods for power delivery, the final solution involved soldering a USB-C cable directly to the output terminals of the step-down (buck) converter. This allowed the processing unit to be powered securely through its native USB-C port, safely facilitating the necessary current flow to the board.

Furthermore, the mechanical assembly required the precise calibration of the LiDAR's spring-loaded leveling bed. As previously detailed in Section 4.1.1, this was necessary to ensure the sensor's measurement plane remained strictly parallel to the ground. The final integrated hardware configuration, with all internal subsystems labeled, is presented in Figure 19.

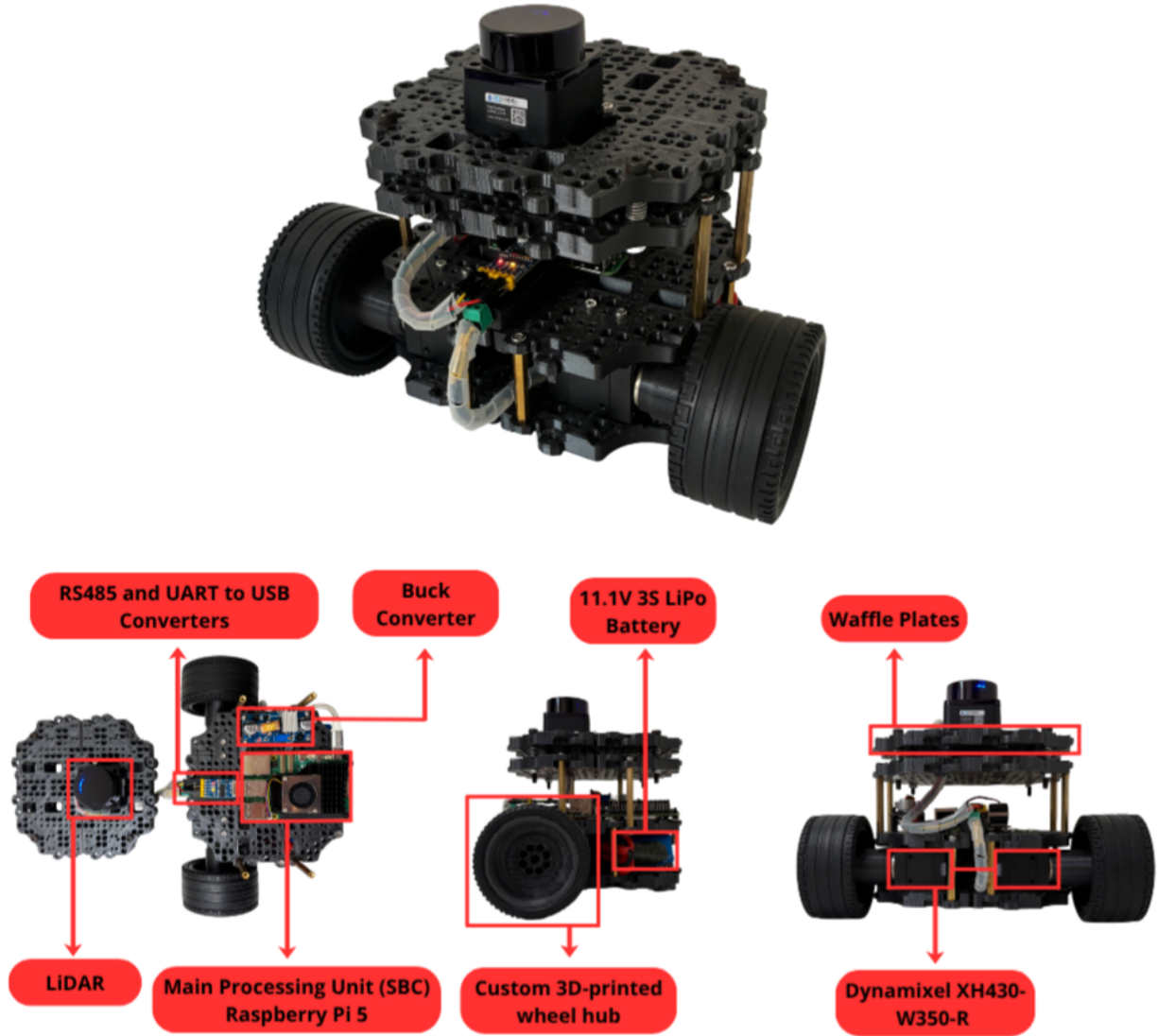


Figure 19: Fully assembled DIY TurtleBot3 highlighting the integrated mechanical and electronic components.

Source: The Author

4.2 Hardware-Software Interface

This section describes the integration between the physical hardware and the software layer. It details the low-level firmware configuration, the containerized operating environment, and the custom ROS 2 nodes developed to bridge the raw sensor and actuation data with the SLAM pipeline.

4.2.1 Low-Level Hardware Configuration

Unlike the standard TurtleBot3 architecture, this project does not utilize an intermediate microcontroller board. Instead, the hardware components interface directly with the Raspberry Pi 5. The two Dynamixel XH430-W350-R servomotors are wired in parallel and connected to the processing unit through a single RS485-to-USB converter, sharing the same serial communication bus. Similarly, the LD19 LiDAR is interfaced directly with the processing unit, utilizing a dedicated UART-to-USB serial adapter to transmit its spatial data.

4.2.2 Microcontroller Firmware

The internal microcontrollers embedded within the Dynamixel actuators required specific firmware parameterization. This configuration was executed using the Dynamixel Wizard software interface ROBOTIS (2024a). Both servomotors were programmed to operate in Velocity Control Mode to properly execute the differential drive kinematics. Furthermore, since both actuators share a single RS485 communication bus, unique hardware identification addresses were flashed into their non-volatile Electrically Erasable Programmable Read-Only Memory (EEPROM), assigning ID 3 to the right motor and ID 4 to the left motor, alongside a matching baud rate of 57600 bps. This specific firmware parametrization prevents data collisions on the shared serial line, ensuring stable and independent control of each wheel.

4.2.3 Operating System and Containerization

The Raspberry Pi hardware platform has official Long-Term Support (LTS) support optimized for Ubuntu 24.04 Canonical Ltd. (2024). However, the specific ROS 2 version utilized in this project natively requires Ubuntu 22.04, as explained in Section 3.1. To resolve this operating system compatibility issue, a Docker container named `slam_container_thesis` was developed using an Ubuntu 22.04 base image. This containerization strategy allows the ROS 2 environment to run properly on the robot without compromising the stability or the updated driver support of the host operating system.

4.2.4 Hardware Interface ROS2 Nodes

The software integration relies on custom ROS 2 nodes to bridge the hardware components with the SLAM pipeline.

For actuation and odometry, a custom Python node (`base_controller`) was developed using the official `dynamixel_sdk`, with its complete implementation open-sourced in the project's repository Cunha (2026b). This node subscribes to the `/cmd_vel` topic, converting linear and angular velocity commands into raw motor instructions. Concurrently, a 20Hz control loop polls the motors' internal magnetic encoders. These readings are processed through the differential drive kinematic model (detailed in Section 2.1.1) to calculate the robot's relative motion. The node publishes this data to the `/odom` topic and broadcasts the `odom` to `base_footprint` transformation within the TF2 system.

For range sensing, the LD19 LiDAR interfaces directly through its UART-to-USB connection. To manage this component, the official ROS 2 driver package provided by the manufacturer ShenZhen LDROBOT Co., LTD (2022b) was utilized. This dedicated node retrieves the raw Direct Time-of-Flight (DToF) point cloud data and publishes it to the `/scan` topic as standard `sensor_msgs/LaserScan` messages. The `/odom` and `/scan` data streams form the complete input dataset consumed by the FastSLAM node.

4.3 Software Architecture and Implementation

This section details the software architecture and the practical implementation of the custom FastSLAM algorithm. It describes the modular C++ package developed for ROS 2, breaking down the system into its core mathematical components and explaining how these routines are orchestrated in real-time by the main execution node.

4.3.1 The Core SLAM Node

This subsection details the software architecture and implementation of the custom FastSLAM algorithm developed for this project. The complete source code, structured as a modular ROS 2 C++ package, is publicly available in the project’s official repository Cunha (2026b). The following text breaks down the system into its core mathematical components, prediction, correction, and mapping, and explains their real-time orchestration by the main execution node.

The prediction phase of the particle filter, implemented in `motion_model.cpp`, is directly derived from the `nav2_amcl` package, specifically its `differential_motion_model.cpp` implementation Open Navigation (2026). This repository was selected as the baseline because it is the official navigation standard for ROS 2. Furthermore, its codebase is a direct mathematical translation of the literature, mirroring the exact theoretical framework from Thrun et al. (2005) detailed in Section 2.1.2 (Motion Model and Odometry).

To integrate this logic into the custom FastSLAM node, the mathematical core was decoupled from the Nav2 architecture and refactored into a standalone C++ class named `MotionModel`. Upon instantiation, the class constructor takes the four kinematic noise parameters ($\alpha_1, \alpha_2, \alpha_3, \alpha_4$) to define the robot’s static variance profile, alongside a Mersenne Twister pseudo-random number generator Matsumoto and Nishimura (1998) required for the sampling step. The dynamic odometry calculated by the Dynamixel encoders is intercepted by the `FastSlamNode` (detailed further below) via a dedicated subscription callback, which iteratively injects the real-time data into the model through the `sampleMotionModel()` method.

The correction phase of the particle filter, implemented in `measurement_model.cpp`, is based on the open-source `openslam_gmapping` repository Grisetti et al. (2007b). This baseline was selected because it is the official implementation of the seminal Rao-Blackwellized Particle Filter optimization algorithms introduced by Grisetti et al. (2007a), representing a historically validated and structurally sound routine for scan evaluation. The codebase translates the Likelihood Field Model framework, as described in Section 2.2.2, into an efficient execution pipeline.

The measurement logic is encapsulated in the `MeasurementModel` class. Upon instantiation, its constructor parameters receive the hit probability (z_{hit}), the measurement variance (σ_{hit}), the random reading probability (z_{rand}), and the maximum operational sensor range (`laser_max_range`) used for the pre-filtering threshold applied within the class, discarding any laser beams with measured distances greater than 98% or less than 15% of the `laser_max_range`.

The integration of this model is managed also by the `FastSlamNode` through its dedicated LiDAR subscription callback. This callback acts as an initial data filter, intercepting and removing anomalous sensor inputs such as infinite values or ranges exceeding the physical sensor threshold. The remaining valid measurement arrays are then injected into the `computeWeight()` method alongside the individual particle's pose hypothesis, its corresponding occupancy grid map, and the angular data of each beam. This routine updates each particle's importance weight based on the spatial correlation between the sensor readings and the current map state.

The mapping phase responsible for generating and maintaining the environment representation, is also derived from the `openslam_gmapping` repository Grisetti et al. (2007b). This functionality is encapsulated within a `GridMapper` object. To update the spatial state, the `updateMap()` method is iteratively invoked, accepting the current grid map, the filtered LiDAR measurements, the estimated particle pose, and the map's angular increment as arguments. This routine implements the log-odds ratio formulation, directly mapping onto the theoretical framework detailed in Section 2.5.1 (Occupancy Grid Maps). By utilizing this inverse sensor model approach, the binary occupancy probability of each cell is updated within each particle's independent map structure.

The core orchestration of the SLAM pipeline is executed by `fastslam_node.cpp`. This node is responsible for integrating the validated baseline classes and managing the execution cycle. Upon initialization, it allocates the predefined number of particles in memory and instantiates the `MotionModel`, `MeasurementModel`, and `GridMapper` object to ensure computational efficiency.

The continuous execution is driven by two primary subscription callbacks:

- **Prediction (Odometry):** By subscribing to the `/odom` topic, the node reacts to displacement data. For every new message, it iterates through the particle cloud and invokes the initialized `MotionModel`, computing the kinematic noise to spatially spread the particles, reflecting movement uncertainty.
- **Correction (LiDAR):** Concurrently, it subscribes to the `/scan` topic. Upon receiving a new environment sweep, the node triggers the correction phase. It iterates through the particles and invokes the `MeasurementModel` to calculate their respective importance weights.

Following the correction step, the node proceeds to evaluate the necessity of a resampling of particles. It calculates the effective number of particles (M_{eff}) to assess the distribution's health, an adaptive resampling approach consolidated in the literature by Grisetti et al. (2007a). If M_{eff} falls below a configured threshold, the node discards particles with negligible weights and duplicating those with high spatial correlation.

In each importance weights update of the particles, the node initiates the mapping phase.

Iterating only through the particle cloud, it invokes the `updateMap()` method of the initialized `GridMapper` instance. This method processes the current scan to update the binary occupancy probabilities of each individual particle's grid. To conclude the cycle, the node identifies the highest-weighted particle (the most probable state hypothesis), extracts its occupancy grid, and publishes it to the `/map` topic, concurrently broadcasting the updated map to `odom` spatial transformation via the TF2 system, as detailed in Section 3.1.2.

4.3.2 Complementary Nodes

Apart from the main SLAM node, complementary nodes were created to manage the robot's movement and to visualize the mapping results.

For path planning, a Python node named `simple_path` was developed utilizing a state-machine architecture. It executes a predefined sequence of geometric primitives (straight lines and arcs), it performs smooth radial turns rather than harsh in-place rotations. It publishes these velocity commands to the `/cmd_vel` topic.

The `base_controller` node (on the physical robot) and the Gazebo simulator (in simulation) are subscribed to this topic, receiving these commands to drive the vehicle at the specified velocity in their respective environments. As feedback, the `simple_path` node subscribes to the `/odom` topic, allowing the robot to navigate to specific coordinates autonomously to gather environmental data. The complete code for this node is available in the project repository Cunha (2026b).

For visualization, the native ROS 2 node `rviz2` is used. It subscribes to the `/map` topic to display the occupancy grid and the particle cloud. An example of a generated map visualized in RViz2 can be seen in Figure 20.

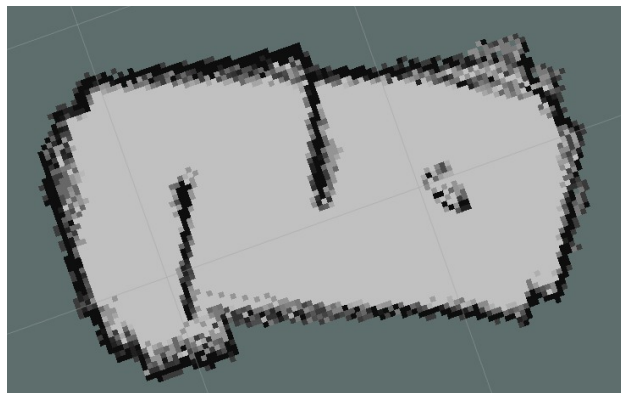


Figure 20: Example of a generated map visualized in RViz2.

4.3.3 Node Structure and Communication Flow

As described in Chapter 2, the FastSLAM node requires two external inputs to produce a map: the robot's odometry for the prediction step, and the LiDAR scan for the correction step. In the ROS 2 architecture, this information arrives through the `/odom` and `/scan` topics.

As explained in Section 4.2.4, on the physical robot, these topics are published by the `base_controller` node and the manufacturer's LiDAR driver, `ldlidar_node`.

Besides the physical robot, the system was structured to support two alternative data sources without changing the core SLAM node. This creates three operational modes:

- **Mode 1 (Physical Robot):** Data comes directly from the hardware sensors.
- **Mode 2 (Gazebo Simulation):** Data is generated by the TurtleBot3 Burger simulation in the Gazebo environment.
- **Mode 3 (ROS 2 Bags):** Data is played back from recorded past runs (`.db3` files).

To provide a clear understanding of how all nodes, topics, and modes interact within the overall architecture, a schematic is presented in Figure 21. This diagram highlights the publishers and subscribers of each node across the different modes.

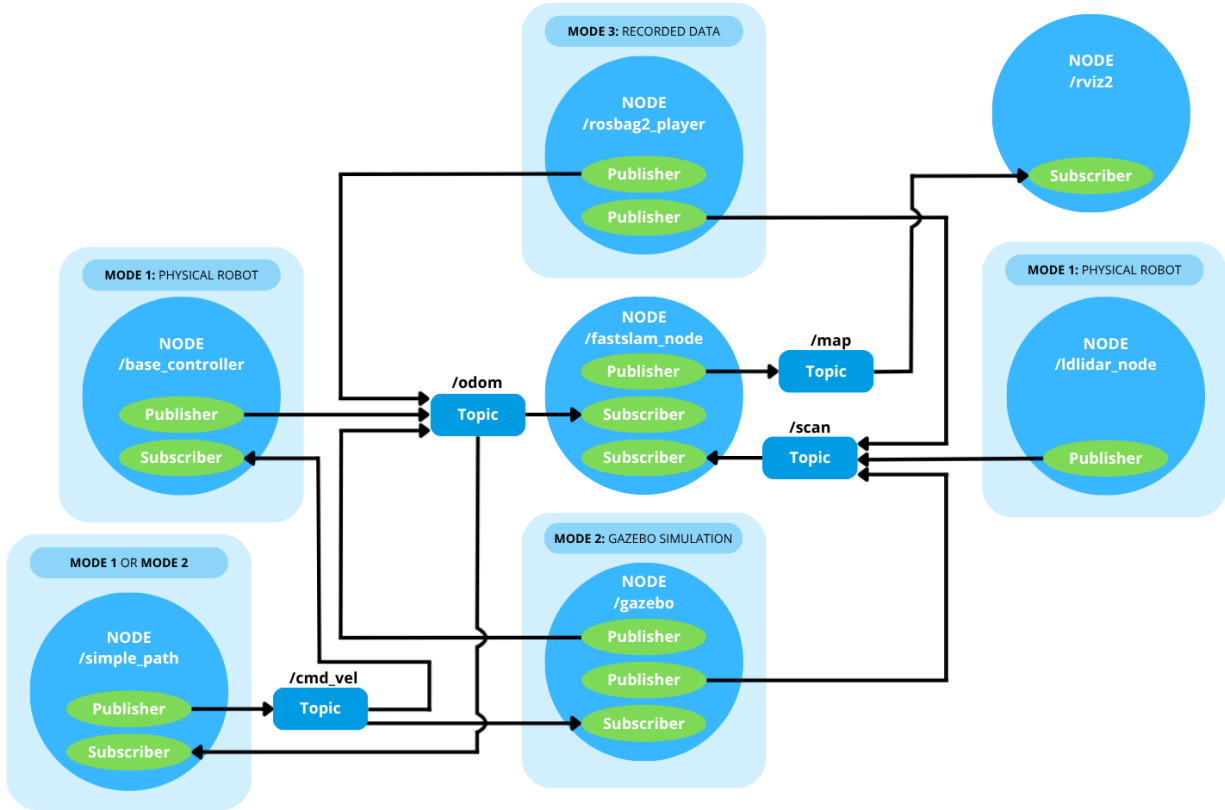


Figure 21: ROS 2 Architecture diagram highlighting nodes, topics, and operational modes.

4.3.4 Human-Machine Interface (HMI)

As detailed in Section 4.3.3, running the complete system requires starting several ROS 2 nodes at the same time. Doing this manually means typing multiple commands in different terminals (Section 3.1.1). To simplify this process, a custom Human-Machine Interface (HMI) was developed using Python. The HMI runs on a separate computer instead of the robot's onboard processor, providing a simple graphical menu with buttons that automatically open the necessary terminals and execute the exact commands for each process. Furthermore, the interface can run the nodes locally for the Gazebo simulation, or use a Secure Shell (SSH) connection to start them directly on the physical robot.

When the HMI is started, an initial window opens (Figure 22) allowing the user to choose between the operating modes defined in Section 4.3.3. Each button navigates to a subsequent window dedicated to starting the specific nodes for the selected mode.

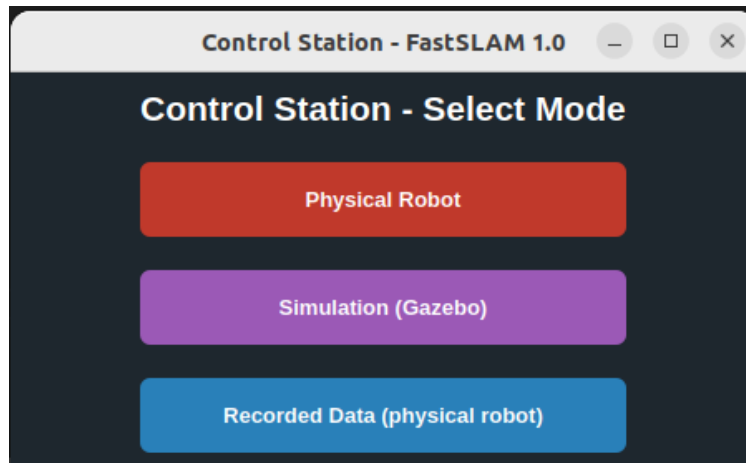


Figure 22: HMI initial mode selection window.

When the "Physical Robot" button is pressed, a second control window opens, as illustrated in Figure 23. First, the user must enter the robot's IP address to establish an SSH connection with the physical hardware. The interface is then divided into several functional groups:

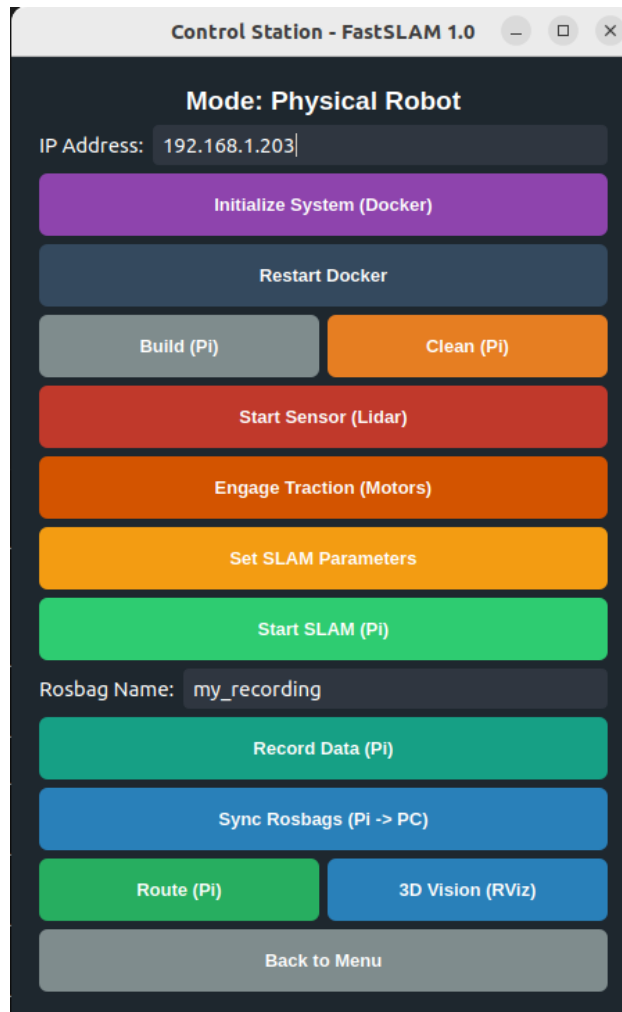


Figure 23: Interface of the Physical Robot control window.

- **System Operations:** The first two buttons are used to initialize and restart the Ubuntu 22.04 Docker container, providing the necessary environment to run the ROS 2 nodes. The “Build (Pi)” button handles the compilation of the ROS 2 nodes, while the “Clean (Pi)” button clears the project directory by erasing data from previous builds.
- **Hardware Initialization:** The “Start Sensor” and “Engage Traction” buttons launch the LiDAR and motor nodes, respectively (explained in Section 4.2.4).
- **SLAM Execution:** The “Set SLAM Parameters” button opens an additional dialog for configuring the SLAM algorithm, which will be discussed later in this section. Once configured, the “Start SLAM” button executes the main mapping node.
- **Data Collection and Navigation:** The “Rosbag Name” text field allows the user to specify a custom filename for the recording, which is initiated by pressing “Record Data”. To execute

autonomous movement, the “Route (Pi)” button starts the node responsible for navigating the robot through the maze (explained in Section 4.3.2).

- **Visualization:** While all the aforementioned nodes execute remotely on the Raspberry Pi via SSH, the final “3D Vision” button launches the RViz2 software locally on the host machine. As mentioned in Section 4.3.3, this displays the resulting map directly on the user’s screen.

When the "Simulation" button is selected, a window similar to the physical robot interface opens, as illustrated in Figure 24. Unlike the physical robot mode, all commands in the simulation environment execute locally on the host machine rather than via SSH. The interface is divided into the following functional groups:

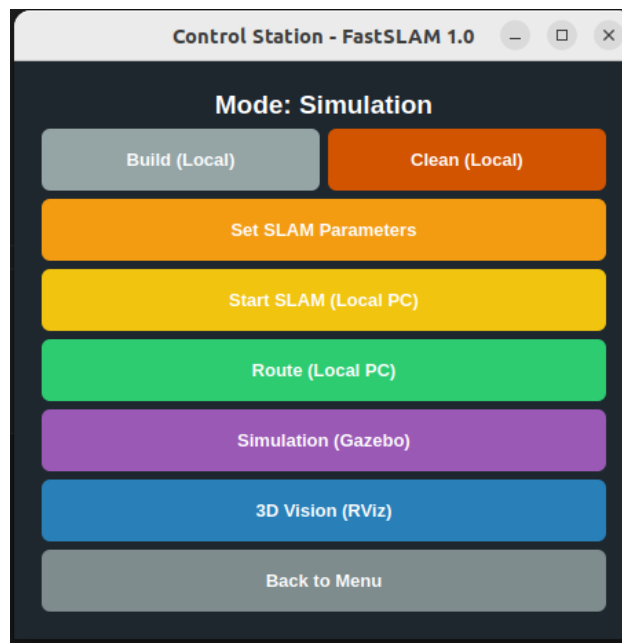


Figure 24: Interface of the Simulation control window.

- **System Operations:** The “Build (Local)” button handles the local compilation of the ROS 2 nodes, while the “Clean (Local)” button clears the project directory by erasing data from previous builds.
- **SLAM Execution:** The “Set SLAM Parameters” button opens an additional dialog for configuring the SLAM algorithm, which will be detailed later in this section. Once configured, the “Start SLAM (Local PC)” button executes the main mapping node locally.
- **Gazebo Simulation:** To open the virtual environment containing the map and the robot model, the “Simulation (Gazebo)” button must be pressed.
- **Autonomous Navigation:** To execute the autonomous movement algorithm, the “Route (Local PC)” button starts the node responsible for navigating the simulated robot through the

virtual maze.

- **Visualization:** The final “3D Vision (RViz)” button launches the RViz2 software locally on the host machine, which displays the resulting map and robot state directly on the user’s screen.

Finally, when the "Recorded Data" mode is selected, the interface transitions to the window illustrated in Figure 25. This mode is designed to execute the SLAM algorithm using previously recorded sensor data (rosbags) from the physical robot, allowing for offline mapping and analysis (as detailed in Section 4.3.3). The interface is organized into the following functional groups:

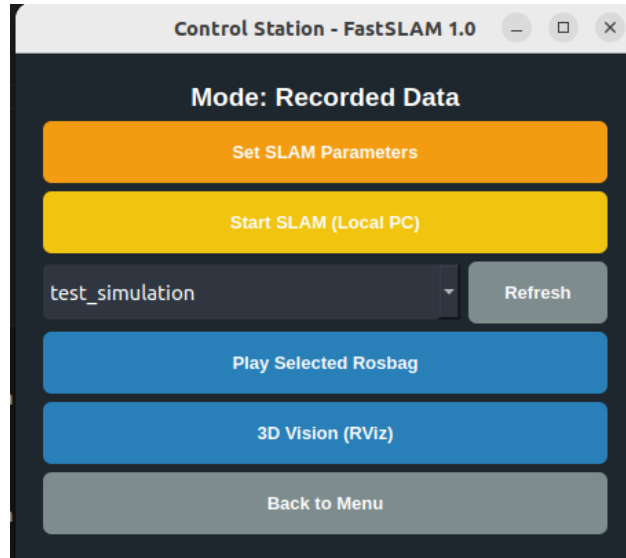
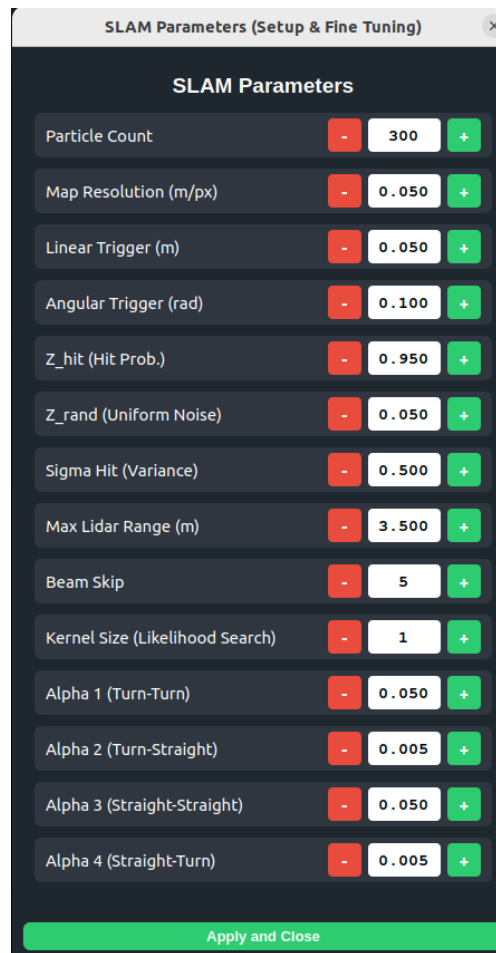


Figure 25: Interface of the Recorded Data control window.

- **SLAM Execution:** The “Set SLAM Parameters” button opens the configuration dialog to tune the SLAM algorithm specifically for the recorded dataset. The “Start SLAM (Local PC)” button then initializes the mapping node on the host machine.
- **Data Playback (Rosbag Player):** A dropdown menu allows the user to select from available historical recordings stored in the workspace, accompanied by a “Refresh” button to update the file list. The “Play Selected Rosbag” button initiates the playback, publishing the stored sensor and odometry topics directly to the ROS 2 network (as explained in the Section 4.3.3).
- **Visualization:** The “3D Vision (RViz)” button launches the RViz2 environment locally, enabling the user to monitor the map generation and trajectory estimation in real time as the recorded data is processed.

As mentioned in the previously in this sections, clicking the “Set SLAM Parameters” button opens the configuration dialog illustrated in Figure 26. This window allows the operator to fine-tune the core FastSLAM algorithm variables prior to execution. The interface exposes the following key

parameter groups:



The image shows a configuration dialog box titled "SLAM Parameters (Setup & Fine Tuning)". It contains a list of parameters, each with a red minus button, a text input field, and a green plus button. The parameters and their current values are:

Parameter	Value
Particle Count	300
Map Resolution (m/px)	0.050
Linear Trigger (m)	0.050
Angular Trigger (rad)	0.100
Z_hit (Hit Prob.)	0.950
Z_rand (Uniform Noise)	0.050
Sigma Hit (Variance)	0.500
Max Lidar Range (m)	3.500
Beam Skip	5
Kernel Size (Likelihood Search)	1
Alpha 1 (Turn-Turn)	0.050
Alpha 2 (Turn-Straight)	0.005
Alpha 3 (Straight-Straight)	0.050
Alpha 4 (Straight-Turn)	0.005

At the bottom of the dialog is a green button labeled "Apply and Close".

Figure 26: SLAM Parameters configuration dialog for algorithm fine-tuning.

- **General and Trigger Settings:** Defines the total Particle Count for the filter distribution (Section 2.4.1), the Map Resolution for the occupancy grid (Section 2.5.1), and the Linear and Angular Triggers. These triggers establish the minimum physical displacement required to compute a new resample of the particles.
- **Measurement Model:** Configures the LiDAR point cloud processing. As detailed in Section 2.2.2, it includes probabilistic weights such as the expected hit probability (z_{hit}) and uniform random noise (z_{rand}), alongside the hit variance (σ_{hit}). It also sets the Max LiDAR Range, the Kernel Size for likelihood searches, and the Beam Skip parameter to downsample the laser scans and reduce computational load.
- **Motion Model:** Adjusts the odometry noise coefficients for the differential drive kinematics, theoretically defined in Section 2.1.2. The variables α_1 through α_4 define the uncertainty

accumulation specific to pure rotations (Turn-Turn), linear translations (Straight-Straight), and the cross-covariance between them (Turn-Straight and Straight-Turn).

Once adjusted, the “Apply and Close” button dynamically passes these variables to the ROS 2 parameter server before initializing the mapping node.

The complete source code for this graphical interface, executed via the `IHM.py` script, is open-source and can be accessed in the project’s official repository Cunha (2026b).

4.4 Experimental Environment and Navigation Trajectory

This section describes the experimental setup used to validate the FastSLAM algorithm. It details the design of a custom-built navigation arena, implemented identically in both the Gazebo simulation and the physical laboratory environment, alongside the predefined trajectory executed by the robot to ensure comprehensive data acquisition.

4.4.1 Environment Design and Structural Features

The evaluation of the FastSLAM algorithm relies on the custom-built enclosed arena detailed in Figure 27. This specific structural layout was implemented identically in both the Gazebo simulation environment and the physical real-world setup. Maintaining a consistent map across both domains allows a direct comparative analysis of the algorithm’s mapping accuracy, sensor noise handling, and odometry drift under different constraints.

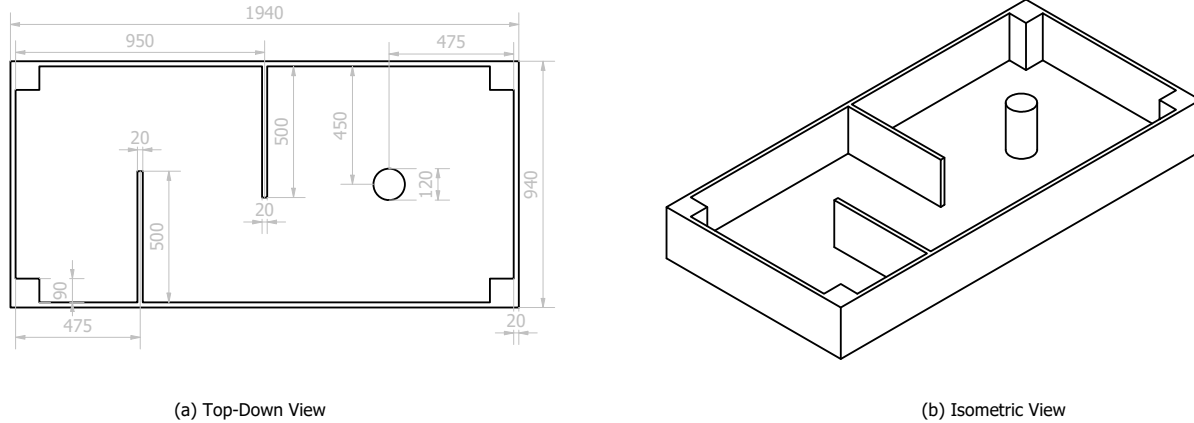


Figure 27: Structural layout and dimensional specifications of the experimental navigation arena: (a) 2D top-down ground truth layout with dimensions in millimeters; (b) 3D isometric projection highlighting the partition walls and cylindrical obstacle.

To replicate this physical arena within the simulation, a custom Gazebo world file was developed using the Simulation Description Format (Simulation Description Format (SDF)). The eXtensible

Markup Language (XML)-based configuration instantiates the exact geometric parameters defined in the ground truth layout. It incorporates static collision meshes for all walls, defining these structural elements as static solid bodies, allowing the simulated TurtleBot3 Burger's LiDAR to interact with the boundaries and publish to the `/scan` topic. The complete source code, named `gazebo_smaller_map.world`, is available in the project's repository Cunha (2026b), as visualized in Figure 28.

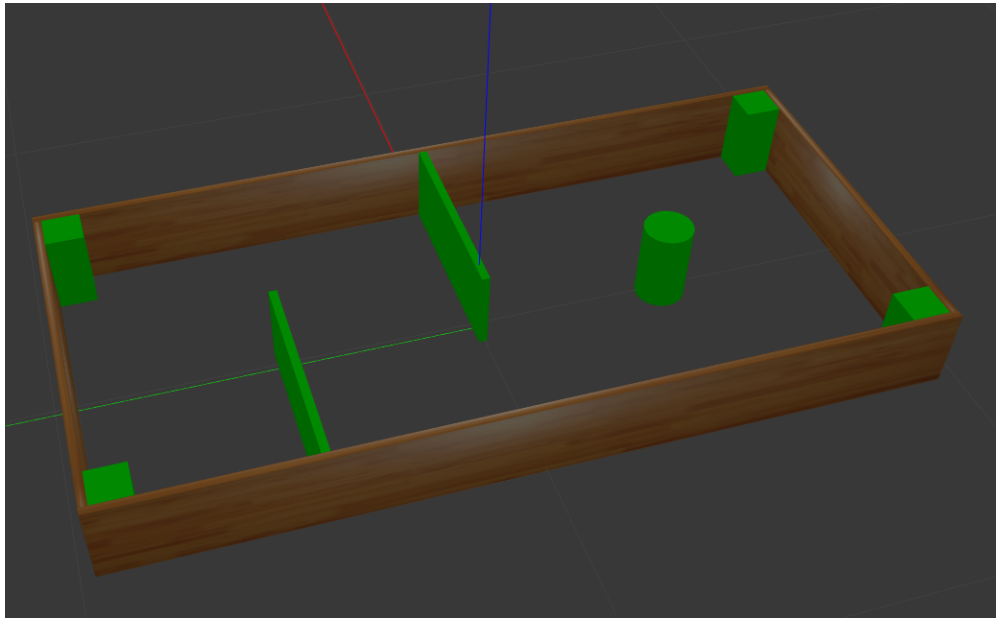


Figure 28: Visual rendering of the custom experimental arena implemented within the Gazebo simulation environment.

The physical testing arena was constructed utilizing a laboratory table from the Starchê robotics team as its foundational base. The outer perimeter consists of wooden plates secured by metal beams at the four corners. To ensure dimensional accuracy and prevent displacement during collisions, the internal partition walls are rigidly anchored using custom 3D-printed support brackets, as modeled in Figure 29. Additionally, the central cylindrical obstacle was fabricated from cardboard. The fully assembled physical environment is presented in Figure 30.

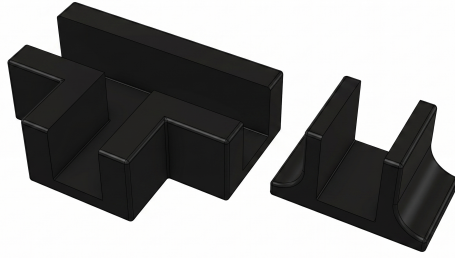


Figure 29: CAD model of the custom 3D-printed support brackets used to secure the internal partition walls.



Figure 30: The fully assembled physical experimental arena in the Starchê team laboratory.

4.4.2 Kinematic Path Planning and Trajectory Execution

To systematically acquire exteroceptive sensor data across the arena, a predefined navigation route was implemented, as illustrated in Figure 31. The trajectory initiates in the lower-left quadrant, proceeds through the initial vertical corridor, executes a radial turn to bypass the first partition wall, traverses the central horizontal section, and concludes with a curved maneuver around the cylindrical obstacle to reach the terminal position on the right side.

This specific path was chosen to ensure comprehensive environmental coverage. Executing this trajectory via the `simple_path` node (detailed in Section 4.3.2) guarantees that the LiDAR captures the complete geometric features necessary to generate an accurate occupancy grid.

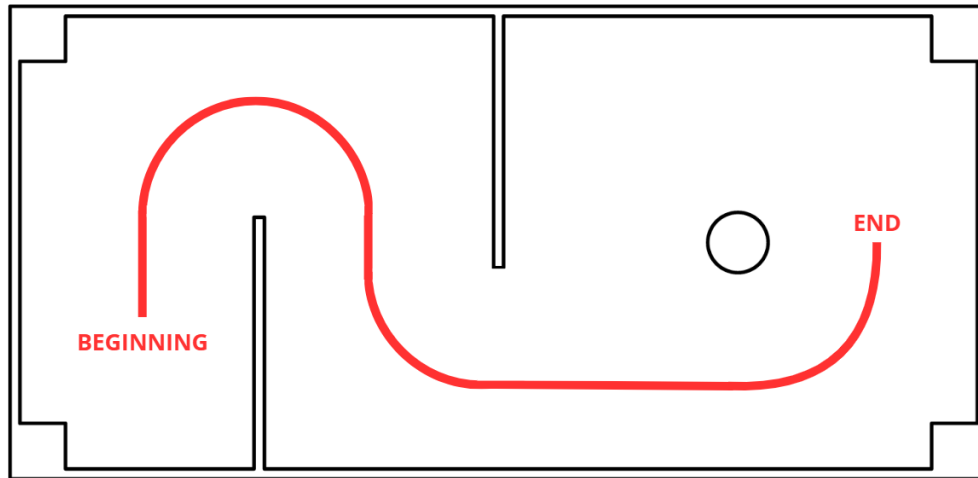


Figure 31: Predefined autonomous navigation trajectory designed to ensure complete LiDAR coverage of the experimental arena.

5 Results and Analysis

This chapter presents the experimental results and quantitative analysis of the FastSLAM algorithm. The performance is assessed in two distinct stages: first, through controlled simulations using the Gazebo environment, and second, through deployment on the physical DIY TurtleBot3 platform. The evaluation focuses on map structural integrity, parameter sensitivity, and the computational efficiency of the SBC hardware, providing an objective comparison between the simulated ground truth and the non-ideal conditions of the real-world environment

5.1 Map Generated in the Simulated Environment

This section evaluates the SLAM algorithm performance in a controlled Gazebo simulation. These tests allow for parameter isolation and accuracy assessment before real-world deployment.

5.1.1 Simulation Execution and Real-Time Visualization

The initial simulation tests were conducted using the Gazebo simulator alongside the RViz2 visualizer. Figure 32 illustrates the real-time mapping execution.

The left side of the figure displays the RViz2 interface, providing a direct look into the state of the ROS 2 topics. The red dots represent the raw LiDAR measurements received via the `/scan` topic. The large red arrow indicates the robot's odometry trajectory, received from the `/odom` topic. The interaction of these topics is further detailed in Section 4.3.3. The generated occupancy grid is published to the `/map` topic and displayed in grayscale. As theoretically defined in Section 2.5.1, these shades represent the probabilistic state of each cell: lighter areas indicate free space, black pixels denote solid boundaries, and the surrounding gray regions represent unknown areas. Additionally, the small axes (colored in red, green, and blue) represent the particle cloud, visually indicating the different pose hypotheses maintained by the FastSLAM filter, as theoretically defined in Section 2.4.1.

On the right side of the figure, the 3D Gazebo simulation environment is shown. It features the TurtleBot3 Burger model actively navigating the virtual arena, with the simulated LiDAR ray-casting represented by the blue beams reading the structural boundaries to generate the sensor data.

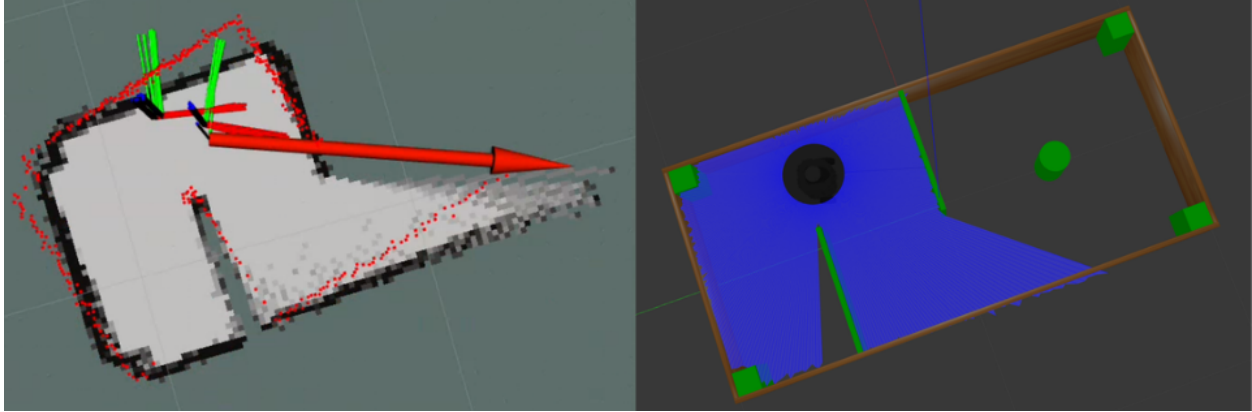


Figure 32: Real-time execution of the FastSLAM algorithm. Left: RViz2 visualization showing the occupancy grid, LiDAR scans (red dots), odometry (large red arrow), and particles (small RGB axes). Right: Gazebo 3D simulation showing the robot and LiDAR ray-casting (blue beams).

5.1.2 Initial Results with Default Parameters

The initial parameters values were configured based on suggestions provided by a conversational AI model, specifically Google's Gemini Google (2026). These baseline values were inputted into the HMI (as detailed in Section 4.3.4) to generate the first map. The parameters were defined as follows:

- **General and Trigger Settings:** Particle Count set to 200 particles; Map Resolution at 0.020 m/px; Linear Trigger at 0.100 m; and Angular Trigger at 0.150 rad.
- **Measurement Model:** Expected hit probability (z_{hit}) at 0.950; uniform random noise (z_{rand}) at 0.150; hit variance (σ_{hit}) at 0.050; Max Lidar Range at 3.000 m; Beam Skip at 1; and Kernel Size at 1.
- **Motion Model:** Odometry noise coefficients set to $\alpha_1 = 0.105$ (Turn-Turn), $\alpha_2 = 0.030$ (Turn-Straight), $\alpha_3 = 0.010$ (Straight-Straight), and $\alpha_4 = 0.005$ (Straight-Turn).

Figure 33 illustrates the initial map exported from RViz2 using these baseline settings. Although the resulting map closely resembles the expected layout of the maze, a visual inspection reveals a discrepancy. When compared to the grid aligned with the robot's initial odometry, the generated map appears to be noticeably tilted.

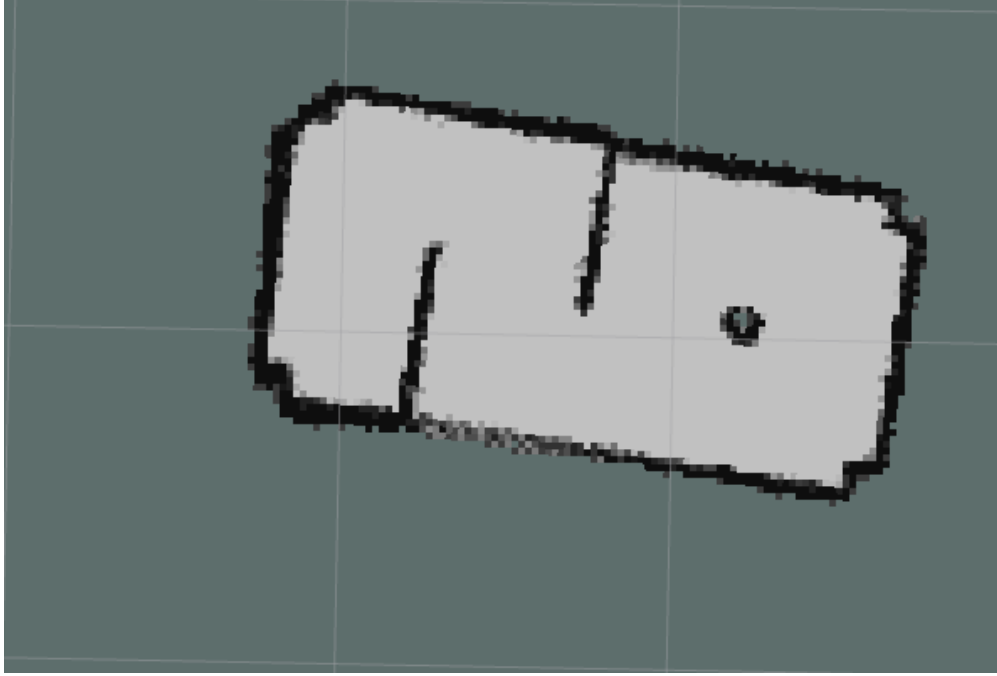


Figure 33: Initial map generated in RViz2. The structural layout resembles the physical maze, but the map exhibits a noticeable tilt relative to the initial odometry grid.

5.1.3 Algorithm Fine-Tuning and Parameter Adjustment

Due to the rotation observed in the initial map, the motion model uncertainty parameters related to rotation (as detailed in Section 2.1.2) were systematically increased: α_1 (Turn-Turn), α_2 (Turn-Straight), and α_4 (Straight-Turn). This adjustment allows the algorithm to tolerate greater odometry errors during maneuvers. Furthermore, the update triggers were increased to allow the particles to spread more widely across the map before a new update occurs, facilitating better scan matching and preventing premature convergence. The complete updated parameter set is defined as follows:

- **General and Trigger Settings:** Particle Count set to 200 particles; Map Resolution at 0.020 m/px; Linear Trigger at 0.150 m (increased 0.050 m); and Angular Trigger at 0.200 rad (increased 0.050 rad).
- **Measurement Model:** Expected hit probability (z_{hit}) at 0.950; uniform random noise (z_{rand}) at 0.050 (decreased 0.100); hit variance (σ_{hit}) at 0.050; Max Lidar Range at 3.500 m (increased 0.500 m); Beam Skip at 1; and Kernel Size at 1.
- **Motion Model:** Odometry noise coefficients set to $\alpha_1 = 0.200$ (Turn-Turn) (increased 0.095), $\alpha_2 = 0.200$ (Turn-Straight) (increased 0.170), $\alpha_3 = 0.050$ (Straight-Straight) (increased 0.040), and $\alpha_4 = 0.100$ (Straight-Turn) (increased 0.095).

The optimized mapping results achieved with this fine-tuning configuration are illustrated in

Figure 34, displaying a corrected alignment relative to the reference grid.

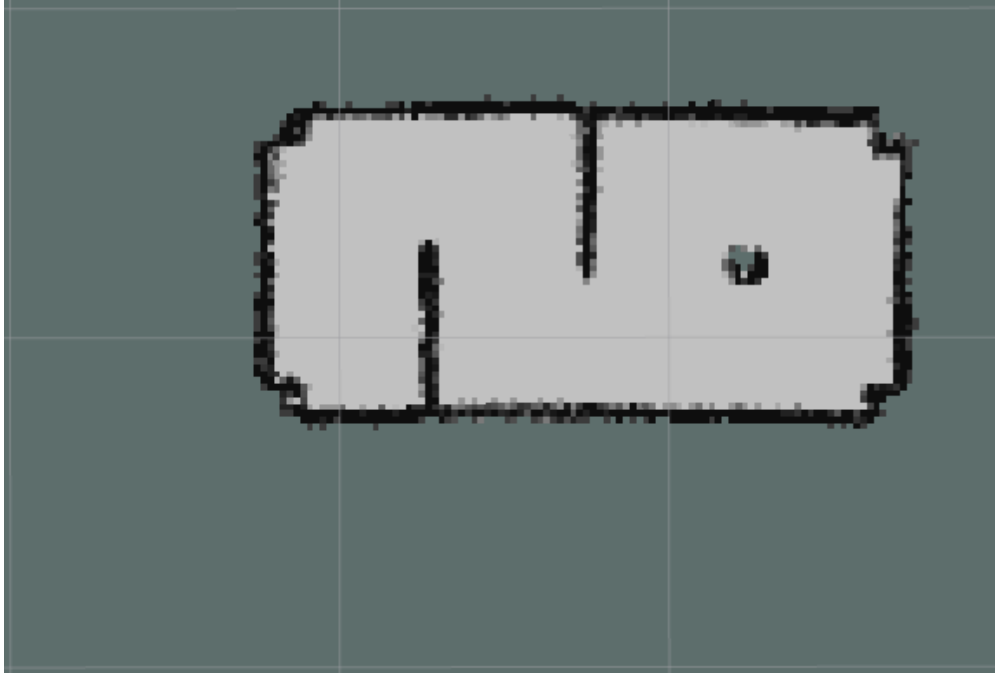


Figure 34: Fine-tuned map generated in RViz2. Increasing the rotation noise coefficients and update triggers resolved the tilting discrepancy, aligning the structural walls correctly.

5.2 Map Generated with the Physical Robot

After validating the SLAM algorithm in simulation, the system was tested on the physical DIY TurtleBot3 platform. This section details the experimental execution, addressing real-world non-idealities. It describes the systematic fine-tuning required to successfully bridge the gap between simulation and real-world deployment.

5.2.1 Physical Robot Execution and Real-Time Visualization

For the physical robot evaluation, tests were conducted using the real sensor data alongside the RViz2 visualizer to monitor the mapping process. Figure 35 illustrates this real-time execution.

The visualization interface follows the exact same structural pattern detailed for the simulation in Section 5.1.1. The occupancy grid (grayscale), the odometry trajectory (large red arrow), and the particle cloud hypotheses (small RGB axes) retain identical representations. The only visual distinction is the rendering of the raw LiDAR measurements from the `/scan` topic, which are displayed as multi-colored dots rather than uniform red dots.

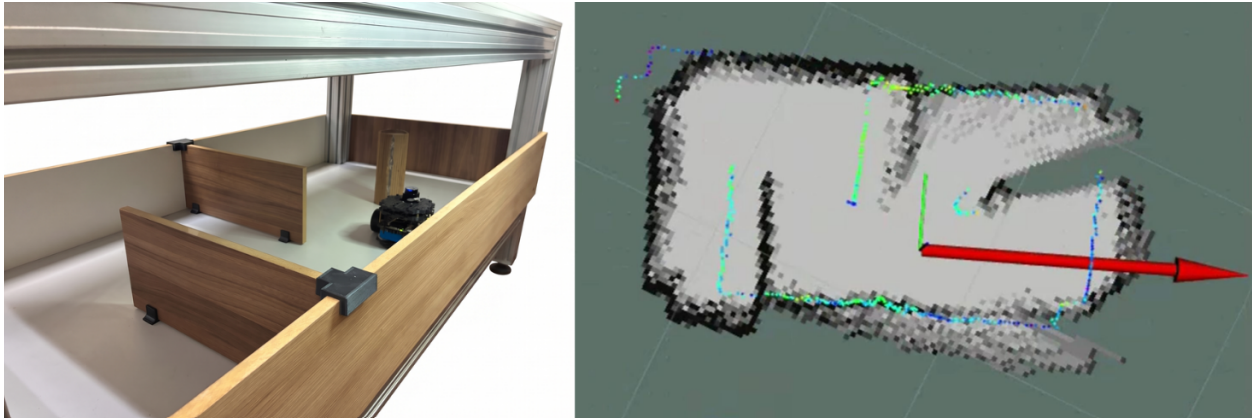


Figure 35: Real-time execution of the FastSLAM algorithm on the physical robot, visualized in RViz2. The LiDAR scans are represented by multi-colored dots.

5.2.2 Initial Results with Simulated Parameters

For the initial physical robot trial, the FastSLAM algorithm was configured using the exact same values applied during the first simulation run, as detailed in Section 5.1.2. Although the Gazebo environment actively injects artificial LiDAR noise (via Gaussian noise models in the sensor plugin) and virtual wheel slippage (governed by the ODE physics engine's friction parameters) to approximate real-world physics Koenig and Howard (2004), these simulated perturbations when compared to the first map result of the real robot in the figure 36 proved to be negligible when compared to the severe non-idealities encountered by the physical DIY TurtleBot3 Burger.

As observed in the resulting map shown in Figure 36, the real-world odometry drift and sensor noise significantly overpowered the baseline parameter constraints. While the algorithm managed to partially construct the overall layout of the maze, the map exhibits a severe tilt and angular misalignment relative to the global coordinate frame.

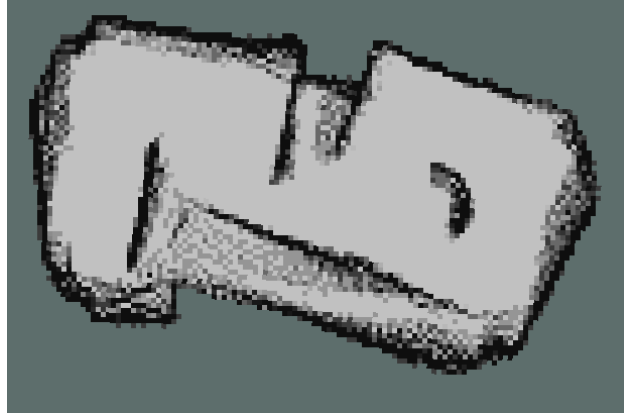


Figure 36: Initial map generated by the physical robot in RViz2 using default parameters. The structural layout partially resembles the physical maze, but the map exhibits a noticeable tilt and distortion relative to the initial odometry grid due to uncompensated real-world drift.

5.2.3 *Sim-to-Real Fine-Tuning and Parameter Adjustment*

It was observed during the run that most of the distortion was caused during the turns of the path shown in Section 4.4.2. For that reason, just like in the simulation tuning (Section 5.1.3), all the motion model uncertainty values related to the turns were increased, along with the two trigger baseline values. Furthermore, the DTOF LD19 LiDAR has a very low uncertainty; therefore, the value z_{rand} , further explained in Section 2.2.2, was substantially decreased.

The final values tested were the exact same as those used for the simulation run, described in Section 5.1.3, and the final map result can be found in Figure 37.

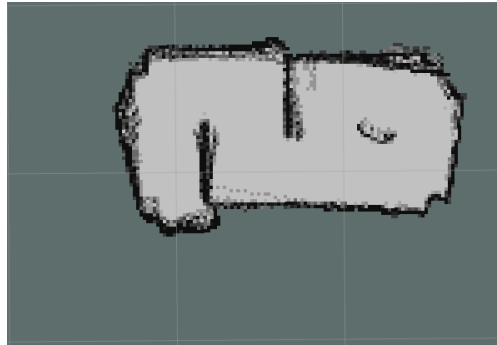


Figure 37: Final optimized map generated by the physical robot in RViz2.

Although the resulting environment representation is not as crisp as the final map generated in the ideal simulation (Figure 34), the systematic parameter tuning demonstrates a substantial increase in overall mapping quality compared to the initial baseline values. The fine-tuning process successfully mitigated the severe angular distortions observed in the first test.

5.3 Quantitative Performance Evaluation

This section evaluates the algorithm's performance through map structural accuracy and hardware efficiency. These metrics provide an objective comparison between simulated and physical results, validating both the occupancy grid integrity and the embedded system's stability.

5.3.1 Quantitative Map Accuracy Analysis

To mathematically validate and compare the structural accuracy of the generated maps, an Occupancy Grid Evaluation was performed against the Ground Truth Floor Plan from Section 4.4.1. The absolute Ground Truth, derived from the 2D CAD blueprint, was precisely scaled utilizing the Gazebo environment's $1\text{m} \times 1\text{m}$ reference grid. This calibrated boundary, represented by the red outline, was then superimposed onto the final occupancy grids of both the simulated and the physical robot runs, aligning their starting origins.

By analyzing the structural overlap, the maximum positional deviation of the mapped walls relative to their true physical coordinates was extracted. For the simulated environment (Figure 38a), the maximum observed divergence was measured at merely 2 pixels. Given the configured map resolution of 0.020 m/px , this translates to a maximum structural error of only 0.04 m (4 cm).

Conversely, the map generated by the physical robot (Figure 38b) exhibited a higher maximum deviation of 9 pixels at its most distorted section. Applying the same resolution, this results in a maximum absolute error of 0.18 m (18 cm).

While the real-world execution presents a considerably higher distortion compared to the ideal simulation, an 18 cm maximum deviation across a nearly 2-meter arena is a highly acceptable margin, since the narrowest corridor has a width more than double this margin and most parts of the map exhibit significantly lower distortion. This quantitative alignment proves that the FastSLAM algorithm successfully maintained the overall topological integrity of the environment, while also leaving room for further tuning by the students of the Starchê team.

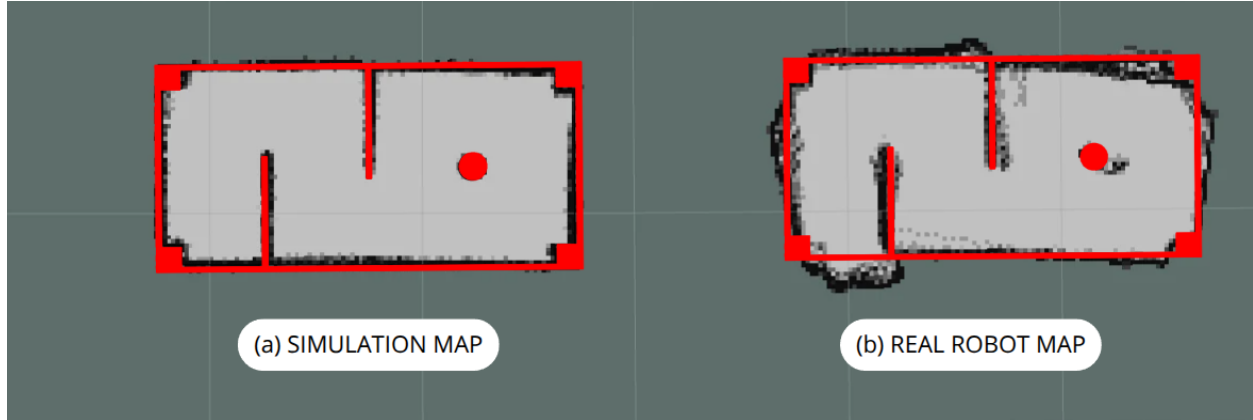


Figure 38: Quantitative structural validation via map overlay. The ground truth CAD boundary (red outline) is superimposed onto the generated maps. (a) The simulation map presents a maximum deviation of 4 cm. (b) The physical robot map presents a maximum deviation of 18 cm.

5.3.2 Computational Performance

To validate the hardware sufficiency claimed in previous sections, a resource utilization analysis (CPU and RAM) of the Raspberry Pi 5 was conducted. To ensure a rigorous evaluation, the FastSLAM algorithm was stress-tested by running 1000 simultaneous particles during the physical mapping phase.

Figure 39 illustrates the resource consumption during operation. According to real-time systems literature Kopetz (2011), processing delays and latency occur when sustained CPU utilization exceeds 80% of total capacity (equivalent to a 320% threshold on a quad-core architecture). During the mapping stress test, the CPU usage peaked at 290%, successfully remaining below the saturation threshold.

Regarding memory allocation, severe system degradation (thrashing) typically occurs when RAM utilization approaches 90% of physical memory Silberschatz et al. (2018). The peak RAM consumption observed during the test was 430 MiB, representing only 5.2% of the 8 GB available. These metrics definitively prove that the selected hardware is fully capable of handling the maximum algorithmic load without computational bottlenecks.

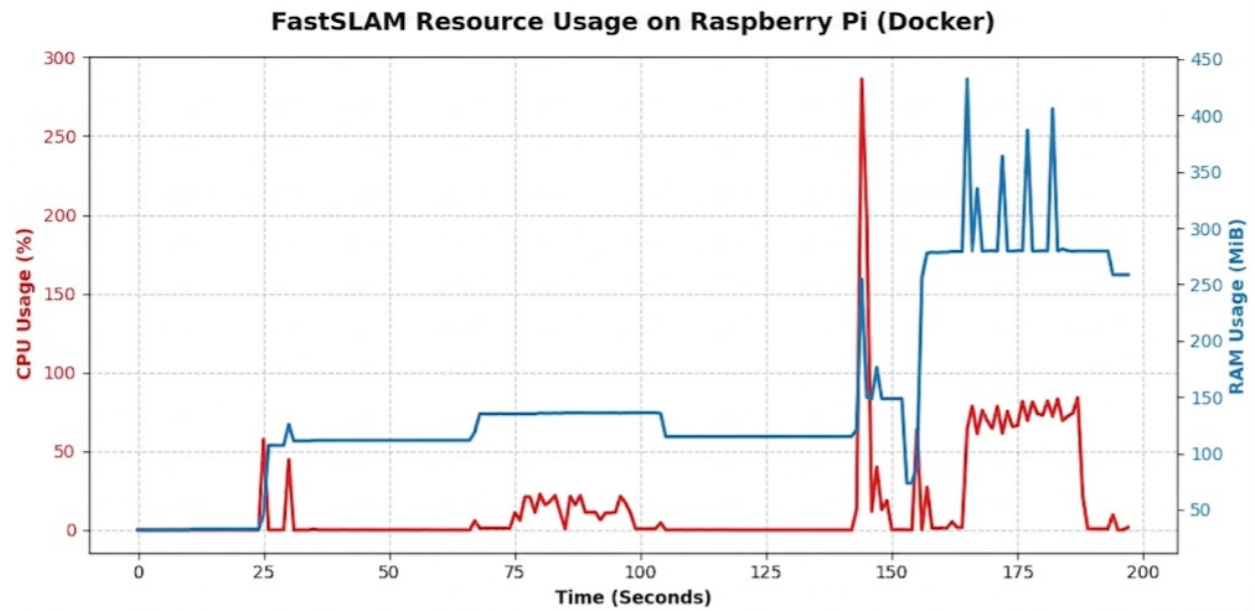


Figure 39: Hardware resource utilization during the 1000-particle stress test. Both CPU and RAM consumption remained strictly below theoretical saturation thresholds.

6 Conclusion

This project successfully developed an open-source framework for mobile robot navigation using a custom-built DIY TurtleBot3 Burger platform integrated with the ROS 2 Humble ecosystem (Cunha, 2026b). This provides the Starchê team with a robust foundation to further understand and implement SLAM technologies. Furthermore, because the entire codebase and hardware specifications are open-source, other universities and research groups can easily replicate the physical robot, utilize the simulation environments, and suggest system improvements that can be continuously integrated back into the repository.

6.1 Summary of Results

The SLAM architecture was built upon widely respected repositories, as detailed in Section 4.3.3. The source code features inline comments cross-referencing this manuscript, allowing students to directly observe the code structure and better understand how the practical SLAM implementation relates to its theoretical foundations.

The physical robot was successfully assembled with an upgraded processing core. The hardware choices for both data acquisition and processing proved highly satisfactory as shown in Section 5. Computational resource monitoring confirmed that the platform easily handles the algorithmic load without reaching saturation, leaving ample processing headroom for the team to analyze different environments.

An experimental map, described in Section 4.4, was created with identical dimensions in both the Gazebo simulation and the physical laboratory. The SLAM algorithm was first validated and tuned in the simulated environment. Subsequently, it was deployed on the physical robot, where there was the necessity of sim-to-real parameter tuning due to real-world non-idealities, such as wheel slippage and odometry drift. By superimposing the generated maps against a ground-truth floor plan, a direct quantitative evaluation of structural accuracy between simulated and physical deployments was achieved, as demonstrated in Section 5.3.1.

The physical robot was successfully assembled, and both the hardware choices for processing and sensor data acquisition proved satisfactory for obtaining the results presented in Section 5. The constructed robot can be utilized by the team to analyze different environments with ample processing headroom.

Finally, utilizing the custom detailed in Section 4.3.4, students can switch between operating the physical robot, running the simulation, and playing back recorded data for offline analysis. Together, these achievements confirm that all specific objectives proposed for this work were successfully fulfilled.

6.2 Future Work

With the framework established for the Starchê Team, future work can directly apply the developed technology. Potential analyses include operating the robot within the constructed map under varying environmental conditions (such as lighting, temperature, and humidity) to evaluate their impact on LiDAR performance. Additional efforts could focus on fine-tuning the SLAM parameters to find optimal values for specific scenarios, investigating how wheel slippage specifically affects robot odometry, and exploring new map layouts with the developed robotic platform.

Furthermore, subsequent studies could implement and evaluate alternative SLAM approaches. This would allow for a comprehensive comparative analysis between the FastSLAM solution developed in this work and other state-of-the-art SLAM algorithms.

7 References

- Alonso, A.G.R., 2025. Sistema de Navegação Autônoma para o Robô Twil. Trabalho de conclusão de curso. Universidade Federal do Rio Grande do Sul (UFRGS). Porto Alegre, RS.
- Arduino Forum Contributors, 2021. Why is differential steering with 2 passive non-caster wheel not used? URL: <https://forum.arduino.cc/t/why-is-differential-steering-with-2-passive-non-caster-wheel-not-used/919564>. accessed: 2026-06-15.
- Boukoberine, N., Zhou, Z., Benbouzid, M., 2019. A critical review on unmanned aerial vehicles power supply and energy management: Solutions, strategies, and prospects. *Applied Energy* 255, 113823.
- BrickLink, 2024. Lego part 44771: Tire 68.8 x 36 zr. <https://www.bricklink.com/v2/catalog/catalogitem.page?P=44771>. Acessado em: 21 abr. 2026.
- Cadena, C., Carlone, L., Carrillo, H., Latif, Y., Scaramuzza, D., Neira, J., Reid, I., Leonard, J.J., 2016. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on Robotics* 32, 1309–1332.
- Canonical Ltd., 2024. Ubuntu on raspberry pi. <https://ubuntu.com/raspberry-pi>. Accessed: 5 June 2026.
- Community, R., 2010. Rep 105: Coordinate frames for mobile platforms. <https://www.ros.org/repos/rep-0105.html>. Accessed: 2025-11-17.
- Cunha, G.S.N., 2026a. 3D pieces for the DIY TurtleBot3 - Starchê UFRGS. <https://www.thingiverse.com/thing:7369630>. Repositório online. Acessado em: 14 jun. 2026.
- Cunha, G.S.N., 2026b. SLAM_Thesis: FastSLAM Implementation in ROS 2. URL: https://github.com/GSNCunha/SLAM_Thesis. acessado em: 16 maio 2026.
- Exchange, R.S., 2025. Slam mapping using rplidar a1 on raspberry pi 4 with ros2 humble. URL: <https://robotics.stackexchange.com/questions/117110/slam-mapping-using-rplidar-a1-on-raspberry-pi-4-with-ros2-humble>. accessed: 2026-04-21.
- Foote, T., 2013. tf: The transform library, in: ROSCon 2013, Stuttgart, Germany. URL: <https://vimeo.com/67980330>.
- Fortune Business Insights, 2024. Slam technology market size, share & covid-19 impact analysis... Disponível em: <<https://www.fortunebusinessinsights.com/slam-technology-market-103347>>.
- Gao, Y., Liu, X., Li, Y., Zhang, H., 2023. A review and bibliometric analysis of the slam algorithm. *Journal of Intelligent & Fuzzy Systems* 45, 5125–5139.
- Google, 2026. Gemini (large language model). <https://gemini.google.com/>. Accessed: 5 June 2026.

- Grisetti, G., Stachniss, C., Burgard, W., 2007a. Improved techniques for grid mapping with rao-blackwellized particle filters. *IEEE transactions on Robotics* 23, 34–46.
- Grisetti, G., Stachniss, C., Burgard, W., 2007b. OpenSLAM Gmapping Open-Source Package. URL: https://github.com/ros-perception/openslam_gmapping. acessado em: 17 maio 2026.
- Koenig, N., Howard, A., 2004. Design and use paradigms for gazebo, an open-source multi-robot simulator, in: 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 2149–2154. doi:10.1109/IROS.2004.1389727.
- Kopetz, H., 2011. Real-Time Systems: Design Principles for Distributed Embedded Applications. 2nd ed., Springer Science & Business Media. Establishes CPU utilization bounds and transient overload thresholds for real-time task scheduling.
- Lages, W.F., Alves, J.A.V., 2017. Differential-drive mobile robot control using a cloud of particles approach. *International Journal of Advanced Robotic Systems* 14, 1–12. doi:10.1177/1729881416680551.
- Lima, J., Costa, P., Silva, E., Petry, M., Moreira, A., 2021. Project-based learning in mechatronics engineering: A case study on a robotics competition team, in: 2021 IEEE Global Engineering Education Conference (EDUCON), pp. 818–825. doi:10.1109/EDUCON47278.2021.9453888.
- Macenski, S., Foote, T., Gerkey, B., Lalancette, C., Woodall, W., 2020. Robot operating system 2: The case for a modular open-source robotics framework for the 21st century, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 10323–10328. doi:10.1109/IROS45743.2020.9341355.
- Matsumoto, M., Nishimura, T., 1998. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation (TOMACS)* 8, 3–30.
- Matthias, K., Kane, S.P., 2015. Docker: Up & Running: Shipping Reliable Containers in Production. O'Reilly Media, Inc.
- Open Navigation, 2026. Nav2 AMCL Component: differential_motion_model.cpp Source Code. URL: https://github.com/ros-navigation/navigation2/blob/main/nav2_amcl/src/motion_model/differential_motion_model.cpp. acessado em: 17 maio 2026.
- Raspberry Pi Ltd, 2023. Raspberry Pi 5 Product Brief. URL: <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>. acessado em: 02 maio 2026.
- Robotics, O., 2022a. Understanding services. <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>. Accessed: 2025-11-17.

- Robotics, O., 2022b. Understanding topics. <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>. Accessed: 2025-11-17.
- ROBOTIS, 2018. Robotis turtlebot burger 3d-printable parts. URL: <https://www.thingiverse.com/thing:3069610>. accessed: 2026-04-21.
- ROBOTIS, 2024a. DYNAMIXEL Wizard 2.0 e-Manual. URL: https://emanual.robotis.com/docs/en/software/dynamixel/dynamixel_wizard2/. acessado em: 16 maio 2026.
- ROBOTIS, 2024b. Dynamixel xh430-w350-r specifications. <https://emanual.robotis.com/docs/en/dxl/x/xh430-w350/>. Acessado em: 21 abr. 2026.
- ROBOTIS, 2024c. Dynamixel xl430-w250-t specifications. <https://emanual.robotis.com/docs/en/dxl/x/xl430-w250/>. Acessado em: 21 abr. 2026.
- ROBOTIS, 2024d. Turtlebot3. Accessed: Oct. 23, 2025. Available: <https://www.robotis.com/service/turtlebot3.php>.
- ROBOTIS, 2026. Turtlebot3 e-manual: Features. <https://emanual.robotis.com/docs/en/platform/turtlebot3/features/>. Acessado em: 21 abr. 2026.
- ShenZhen LDROBOT Co., LTD, 2022a. DTOF LiDAR LD19 Specification, V1.0. URL: https://github.com/LetsOKdo/ld19/blob/master/datasheet/LDROBOT_LD19_Datasheet_V1.0.pdf. acessado em: 02 maio 2026.
- ShenZhen LDROBOT Co., LTD, 2022b. Ldrobot ros2 lidar node package. URL: https://github.com/ldrobotSensorTeam/ldlidar_stl_ros2. acessado em: 16 maio 2026.
- Siciliano, B., Khatib, O. (Eds.), 2016. Springer Handbook of Robotics. Second ed., Springer-Verlag, Berlin, Heidelberg.
- Siegwart, R., Nourbakhsh, I.R., Scaramuzza, D., 2011. Introduction to Autonomous Mobile Robots. Second ed., MIT Press, Cambridge, MA.
- Silberschatz, A., Galvin, P.B., Gagne, G., 2018. Operating System Concepts. 10th ed., John Wiley & Sons. Defines memory management, paging thresholds, and thrashing limits in modern operating systems.
- Thrun, S., Burgard, W., Fox, D., 2005. Probabilistic Robotics. MIT Press, Cambridge, MA.
- Universidade Federal do Rio Grande do Sul (UFRGS), M., 2024. Move - mostra de veículos e equipes de competição. Disponível em: <<https://www.ufrgs.br/move/>>. Acesso em: 23 out. 2025.
- XLSEMI, 2012. XL4015 5A 180KHz 36V Buck DC to DC Converter Datasheet. URL: https://datasheet.lcsc.com/szlcsc/1811081616_XLSEMI-XL4015E1_C51661.pdf. acessado em: 02 maio 2026.

Zbotic, 2024. Slam with raspberry pi: Turtlebot3 navigation and mapping. URL: <https://zbotic.in/slam-raspberry-pi-turtlebot3-navigation-mapping/>. accessed: 2026-04-21.